

1. Structure avec Map
2. Parcours en profondeur
3. Parcours en largeur

Structure de données avancée pour les graphes

Nous considérons une nouvelle structure de données pour représenter un graphe de n sommets et m arêtes :

- Une **carte d'adjacence** : on utilise *Map* (plutôt qu'une liste) *pour les arêtes de chaque sommet*, où la clé d'accès est le sommet.

👉 Cela permet d'accéder à une arête spécifique (u, v) en $O(1)$ en espérance.

Map d'adjacence

Structure de base. Pour permettre d'améliorer `getEdge(u, v)`, on utilise une *Map* pour les arêtes propres à chaque sommet.

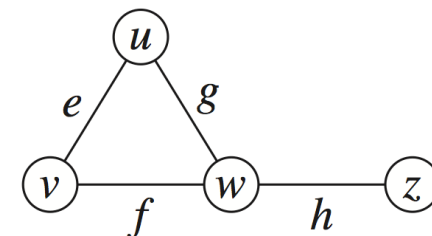
- l'autre extrémité de l'arête est utilisée comme **clé** et la référence à l'instance *Edge* comme **valeur**

Pas de changement dans l'espace : $O(n+m)$.

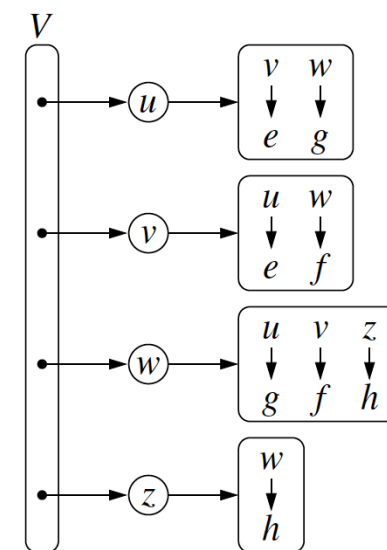
Structure augmentée. Comme pour la liste d'adjacence, pour assurer que `edges()` et `numEdges()` soient efficaces, on peut maintenir une liste auxiliaire pour les arêtes.

Objet de type *Edge* augmenté

- même principe que pour liste d'adjacence



Opérations pour n sommets et m arêtes	Temps d'exécution (pire cas)
<code>numVertices()</code> , <code>numEdges()</code>	$O(1)$
<code>vertices()</code>	$O(n)$
<code>edges()</code>	$O(m)$
<code>getEdge(u, v)</code>	$O(1)$ en espérance
<code>outDegree(v)</code> , <code>inDegree(v)</code>	$O(1)$
<code>outgoingEdges(v)</code> , <code>incomingEdges(v)</code>	$O(deg(v))$
<code>insertVertex(x)</code>	$O(1)$
<code>insertEdge(u, v, x)</code>	$O(1)$ en espérance
<code>removeEdge(e)</code>	$O(1)$ en espérance
<code>removeVertex(v)</code>	$O(deg(v))$



Implémentation de AdjacencyMapGraph

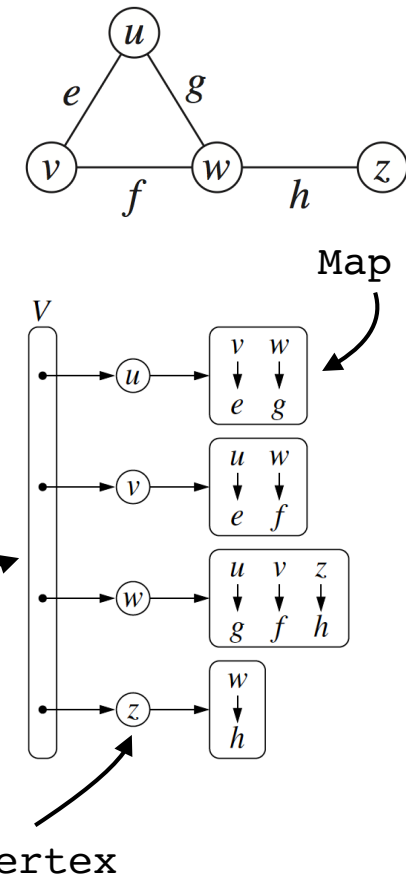
On utilise des `PositionalList` pour les listes de sommets et arêtes et pour chaque sommet une `Map` pour les arêtes sortantes et entrantes avec comme clés les sommets adjacents et comme valeurs les arêtes propres.

On parcourt tous les sommets et les arêtes en parcourant les listes positionnelles correspondantes.

voir le code :

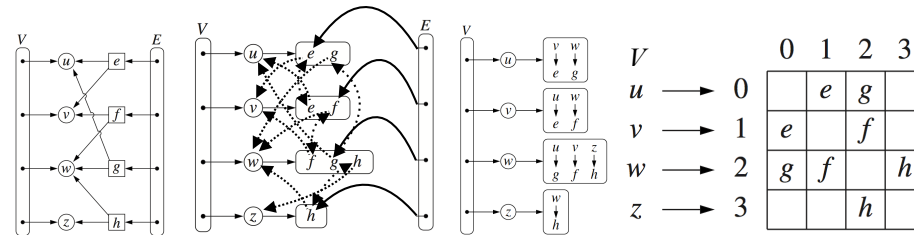
`AdjacencyMapGraph.java`

`LinkedPositionalList`



Le type retourné par `getOutgoing` et `getIncoming` dans la classe `InnerVertex` est `Map`

Résumé des performances en pire cas des opérations sur les graphes



- n sommets, m arêtes
- pas d'arêtes parallèles
- pas d'auto-loop

	Liste d'arêtes	Liste d'adjacence	Dictionnaire d'adjacence	Matrice d'adjacence
Espace	$O(n + m)$	$O(n + m)$	$O(n + m)$	$O(n^2)$
<code>numVertices()</code>	$O(1)$	$O(1)$	$O(1)$	$O(1)$
<code>numEdges()</code>	$O(1)$	$O(1)$	$O(1)$	$O(1)$
<code>vertices()</code>	$O(n)$	$O(n)$	$O(n)$	$O(n)$
<code>edges()</code>	$O(m)$	$O(m)$	$O(m)$	$O(m)$
<code>getEdge(u, v)</code>	$O(m)$	$O(\min(d_u, d_v))$	$O(1)$ esp.	$O(1)$
<code>outDegree(v),</code> <code>inDegree(v)</code>	$O(m)$	$O(1)$	$O(1)$	$O(n)$
<code>outgoingEdges(v),</code> <code>incomingEdges(v)</code>	$O(m)$	$O(d_v)$	$O(d_v)$	$O(n)$
<code>insertVertex(x)</code>	$O(1)$	$O(1)$	$O(1)$	$O(n^2)$
<code>removeVertex(v)</code>	$O(m)$	$O(d_v)$	$O(d_v)$	$O(n^2)$
<code>insertEdge(u, v, x)</code>	$O(1)$	$O(1)$	$O(1)$ esp.	$O(1)$
<code>removeEdge(e)</code>	$O(1)$	$O(1)$	$O(1)$ esp.	$O(1)$

Parcourir les noeuds des graphes

Parcourir/traverser un graphe est une procédure systématique pour visiter tous les sommets et toutes les arêtes d'un graphe. Un parcours est efficace s'il visite tous les sommets et toutes les arêtes dans un temps proportionnel à leurs nombres, c'est-à-dire en temps linéaire.

Les algorithmes pour parcourir un graphe sont essentiels pour répondre à des questions fondamentales impliquant la notion d'accessibilité, c'est-à-dire, pour “voyager” d'un sommet à un autre en parcourant les chemins du graphe.

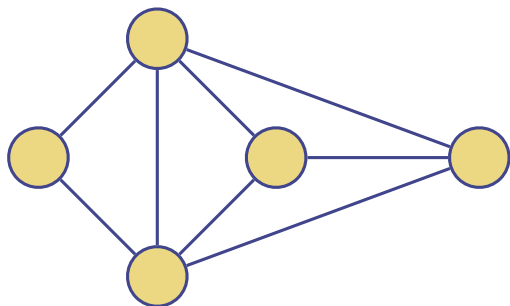
Dans la suite de ce module, on présente deux algorithmes efficaces pour parcourir un graphe : la recherche en profondeur et la recherche en largeur.

Recherche en profondeur (DFS)

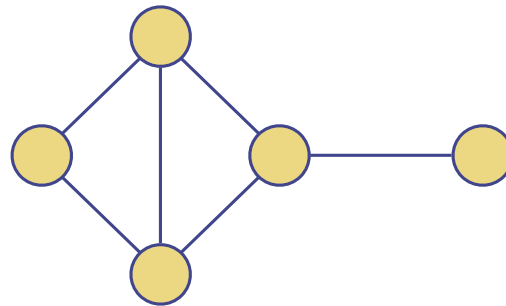
La recherche en profondeur (DFS : Depth-First-Search) est une technique générale pour traverser un graphe. Le parcours DFS d'un graphe G :

- Visite tous les sommets et arêtes de G
- Détermine si G est connexe
- Calcule les composantes connexes de G
- Calcule une forêt couvrante de G

La DFS sur un graphe avec n sommets et m arêtes s'exécute dans $O(n + m)$

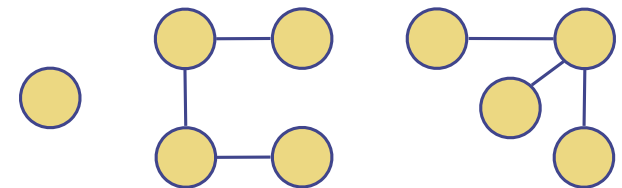


Graphe connexe



Graphe connexe

- Une **forêt couvrante** d'un graphe G est un sous-graphe couvrant de G (càd contenant tous ses sommets) et ne contenant aucun cycle. Ses composantes connexes sont des arbres (un arbre par composante connexe)
- Si G est connexe, une forêt couvrante est alors un **arbre couvrant**
- Si G est lui-même un arbre, alors sa seule forêt couvrante est G lui-même



Forêt

Algorithme DFS

La DFS dans un graphe G est analogue à se promener dans un labyrinthe avec une ficelle et une boîte de peinture.

On débute à un point de départ, par exemple le sommet s , qu'on initialise en y fixant une extrémité de notre ficelle et en le peignant "visité".

Le sommet s est le sommet "courant", $u = s$.

On traverse G en considérant une arête (arbitraire) adjacente à u , (u, v) . Si le sommet v est déjà peint "visité", on ignore cette arête. Si, au contraire, v n'a pas été visité, alors on y va en déroulant notre ficelle. Arrivé à v , on le peint "visité", et on en fait notre sommet "courant".

Éventuellement, on arrive à une "impasse", c'est-à-dire à un sommet pour lequel toutes les arêtes adjacentes mènent à des sommets déjà visités. Pour se sortir de cette impasse, on suit notre ficelle qui nous a mené à v vers l'arrière (on "backtrack") pour revenir à u . On continue à explorer les arêtes adjacentes à u qu'on a pas encore considérées. Si toutes ses arêtes conduisent à des sommets déjà visités, on utilise le même truc pour retourner au sommet précédent, et on poursuit notre quête !

On continue ce "backtracking" jusqu'à ce qu'on trouve un sommet qui a une ou des arêtes encore inexplorées. On continue ainsi jusqu'à ce qu'on revienne au sommet de départ s et que toutes les arêtes aient été explorées.

Algorithme DFS(G, u):

Input: Un graphe G et un sommet u de G

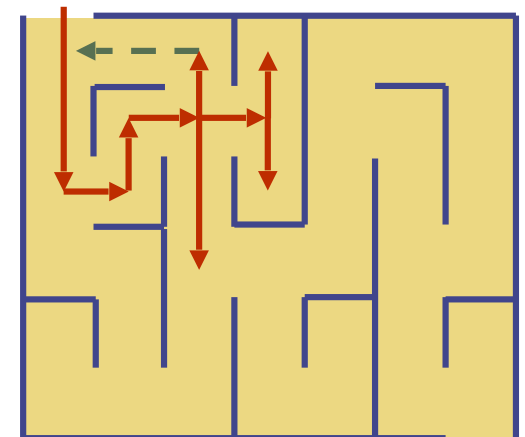
Output: Une collection de sommets accessibles de u et leurs arêtes y menant

pour toute arête sortante $e = (u, v)$ de u **faire**

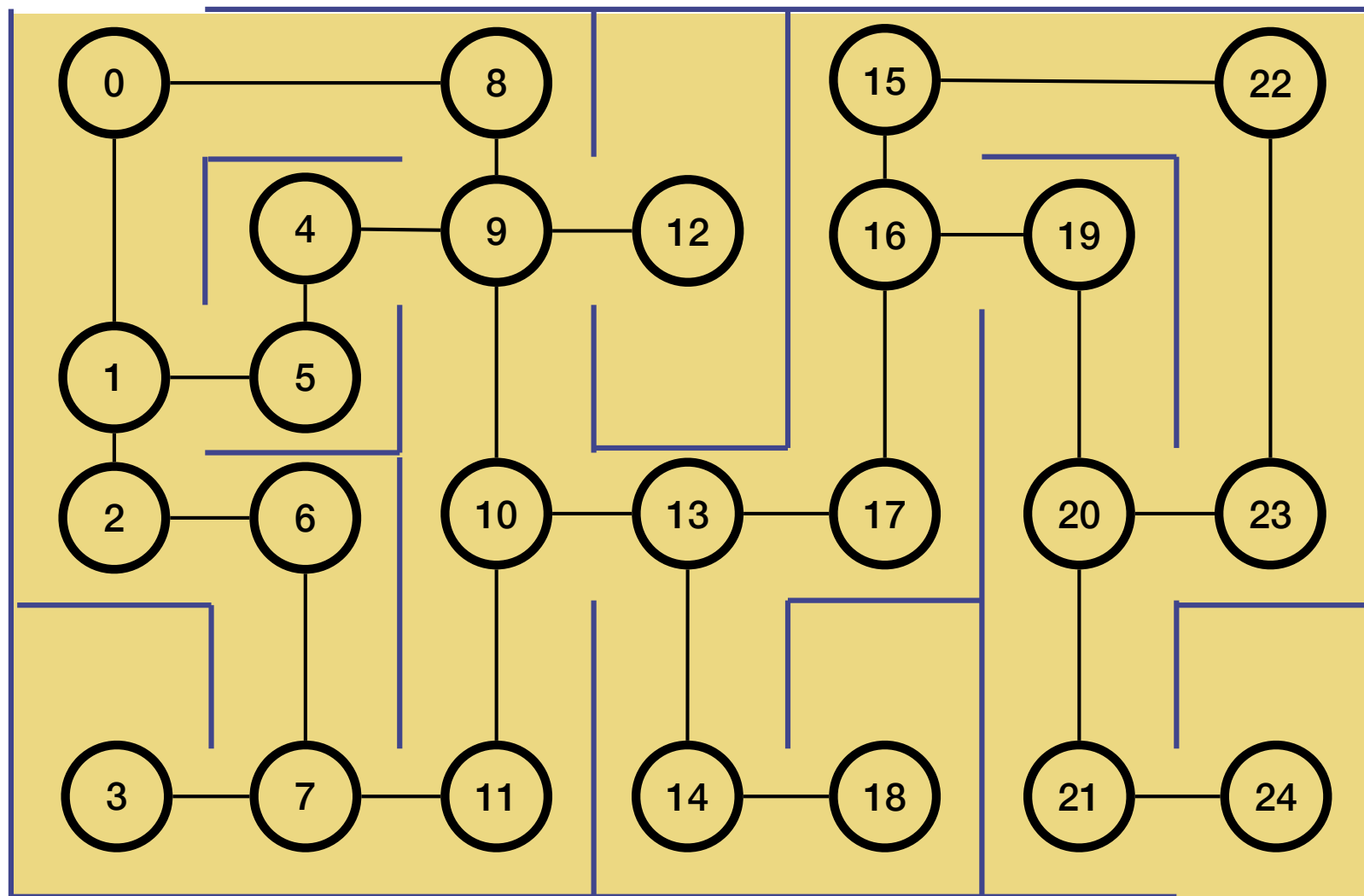
si sommet v n'a pas été visité **alors**

 marquer sommet v visité (via l'arête e)

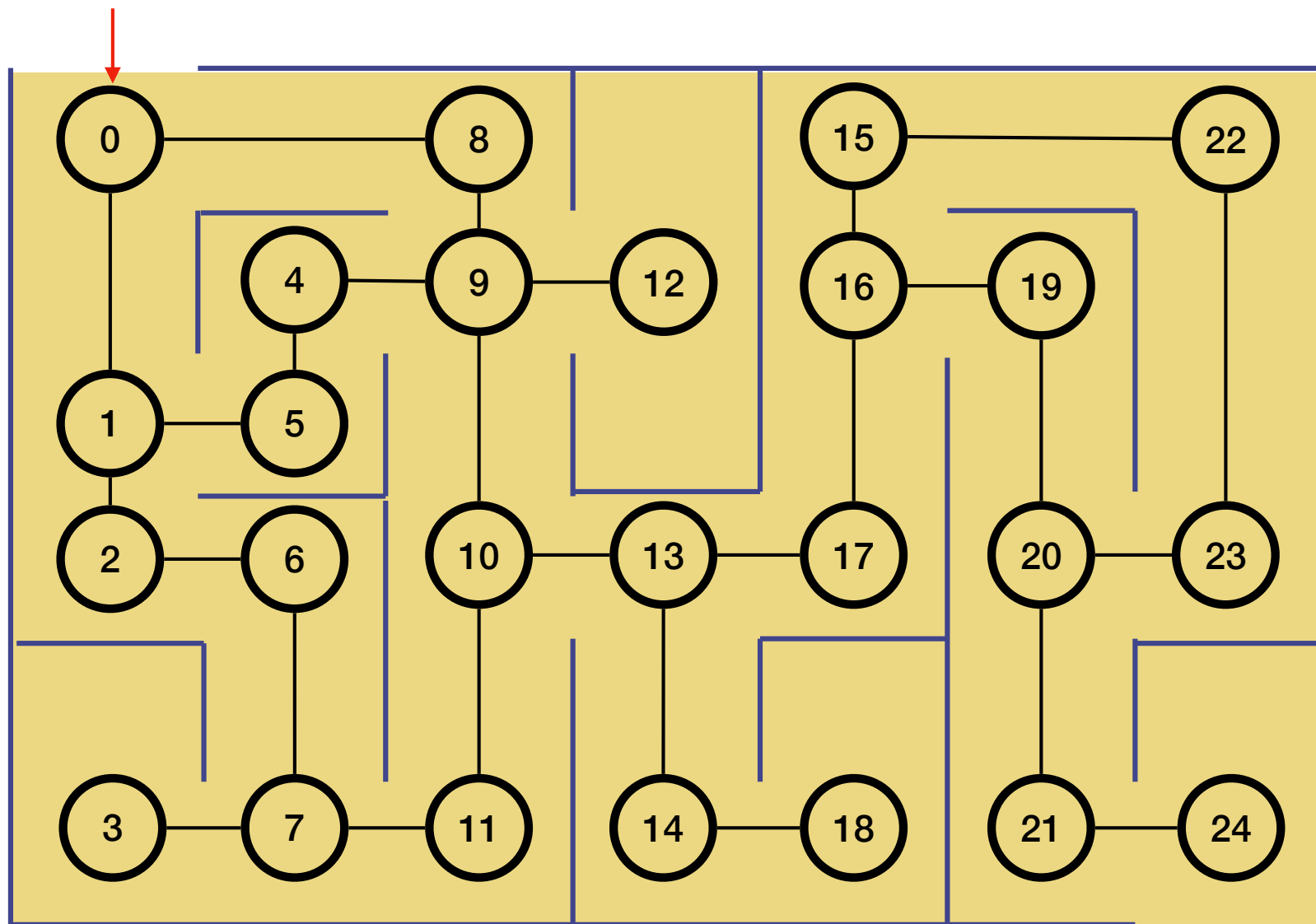
 appeler récursivement DFS(G, v)



DFS

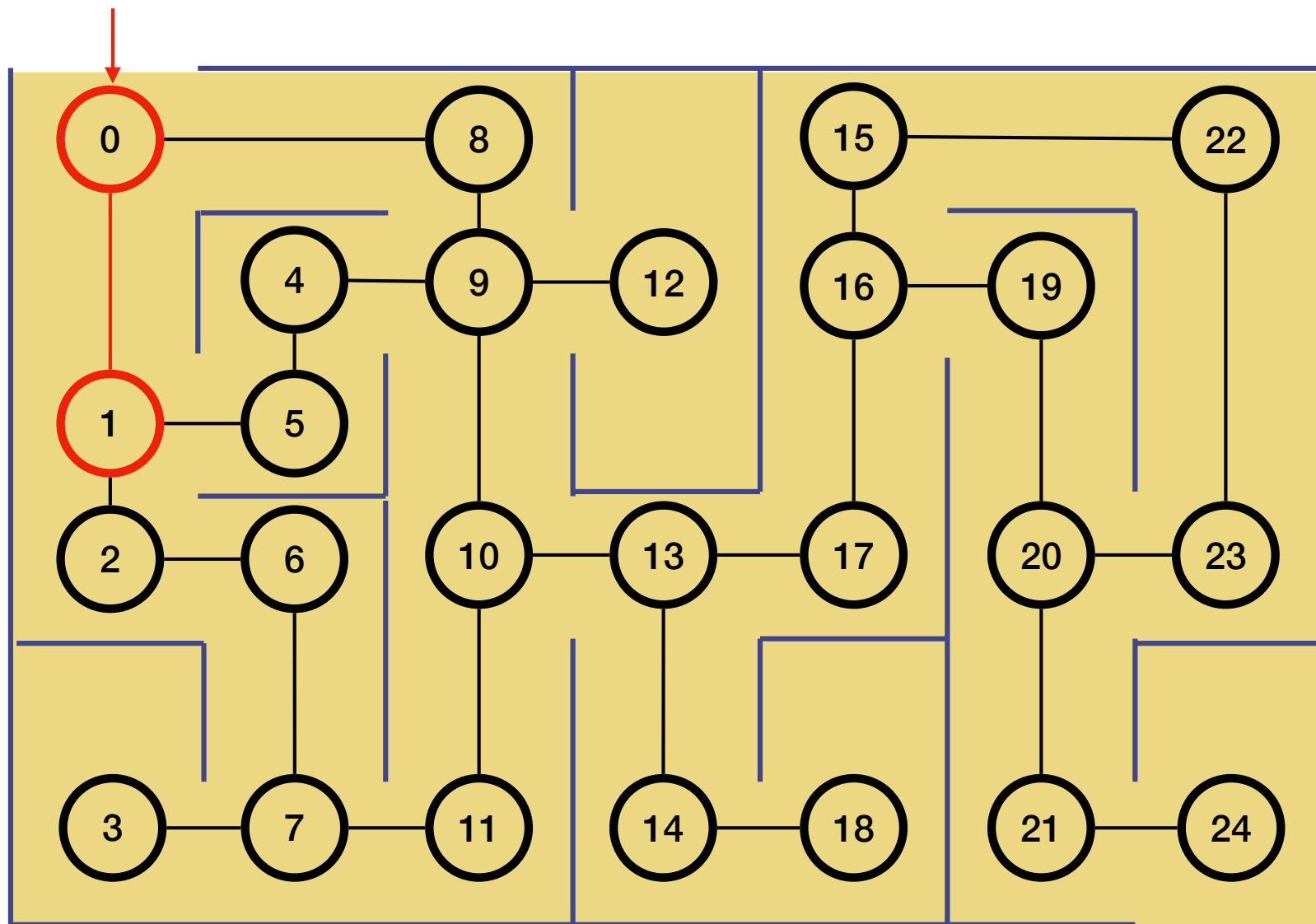


DFS

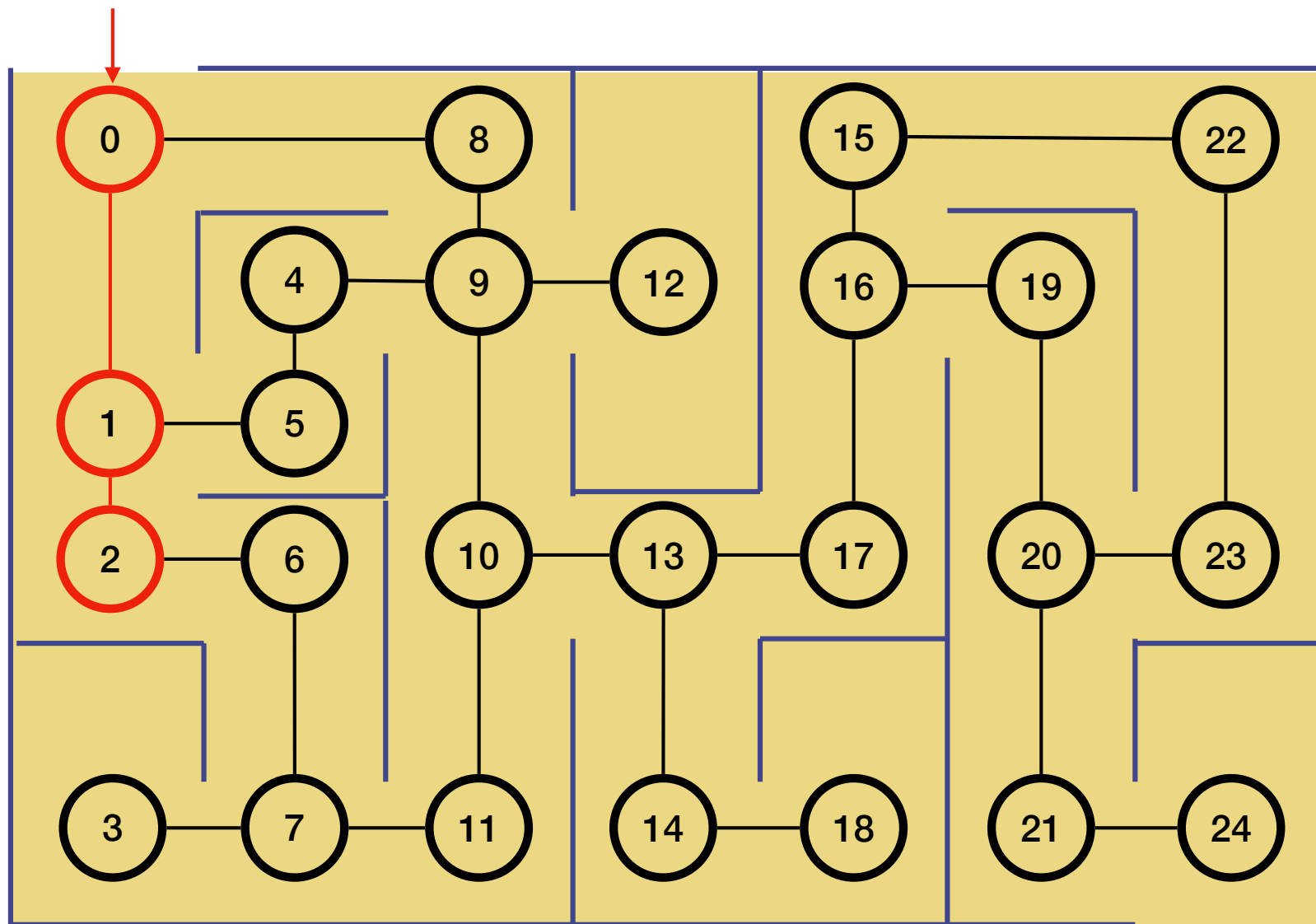




DFS

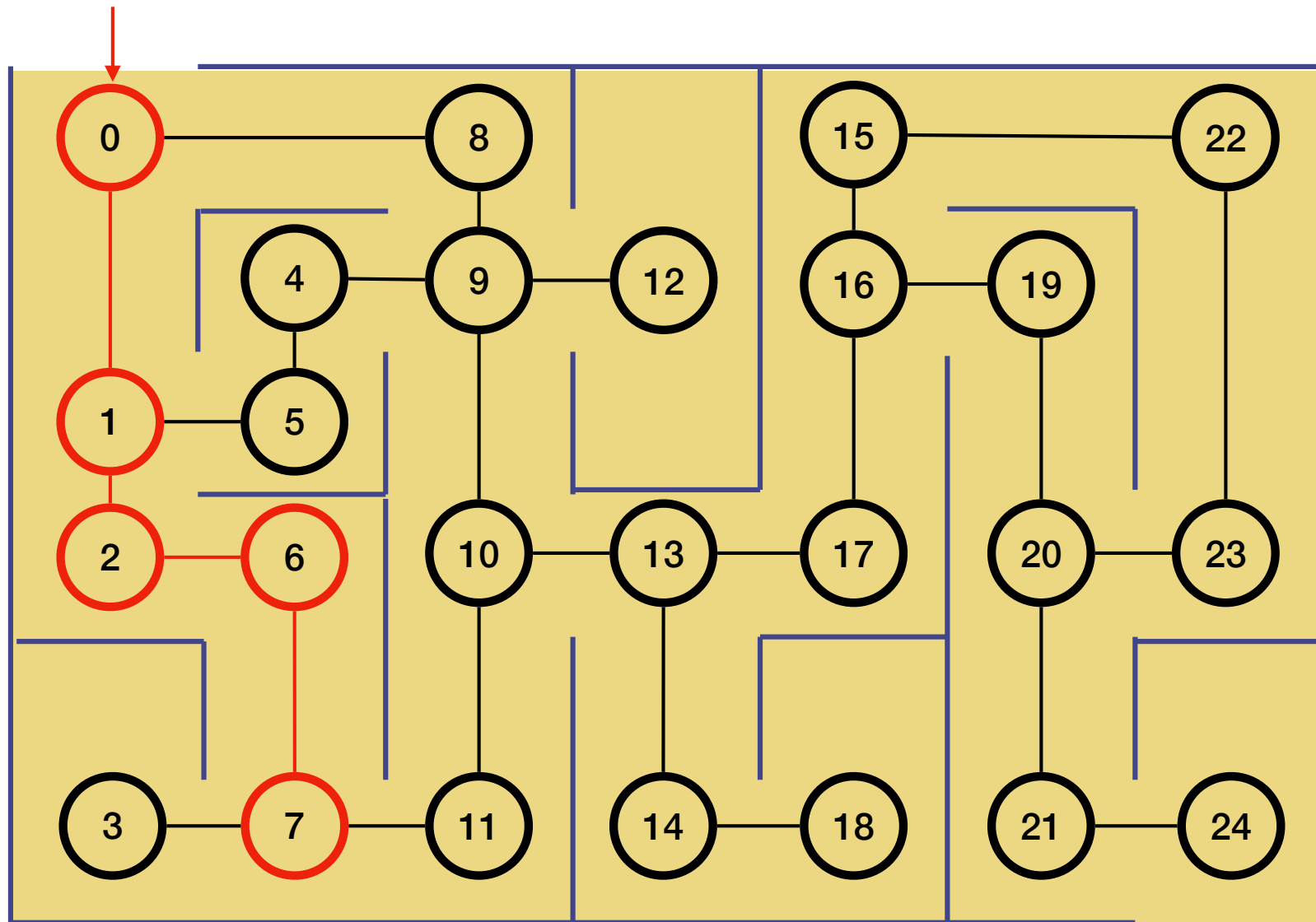


DFS

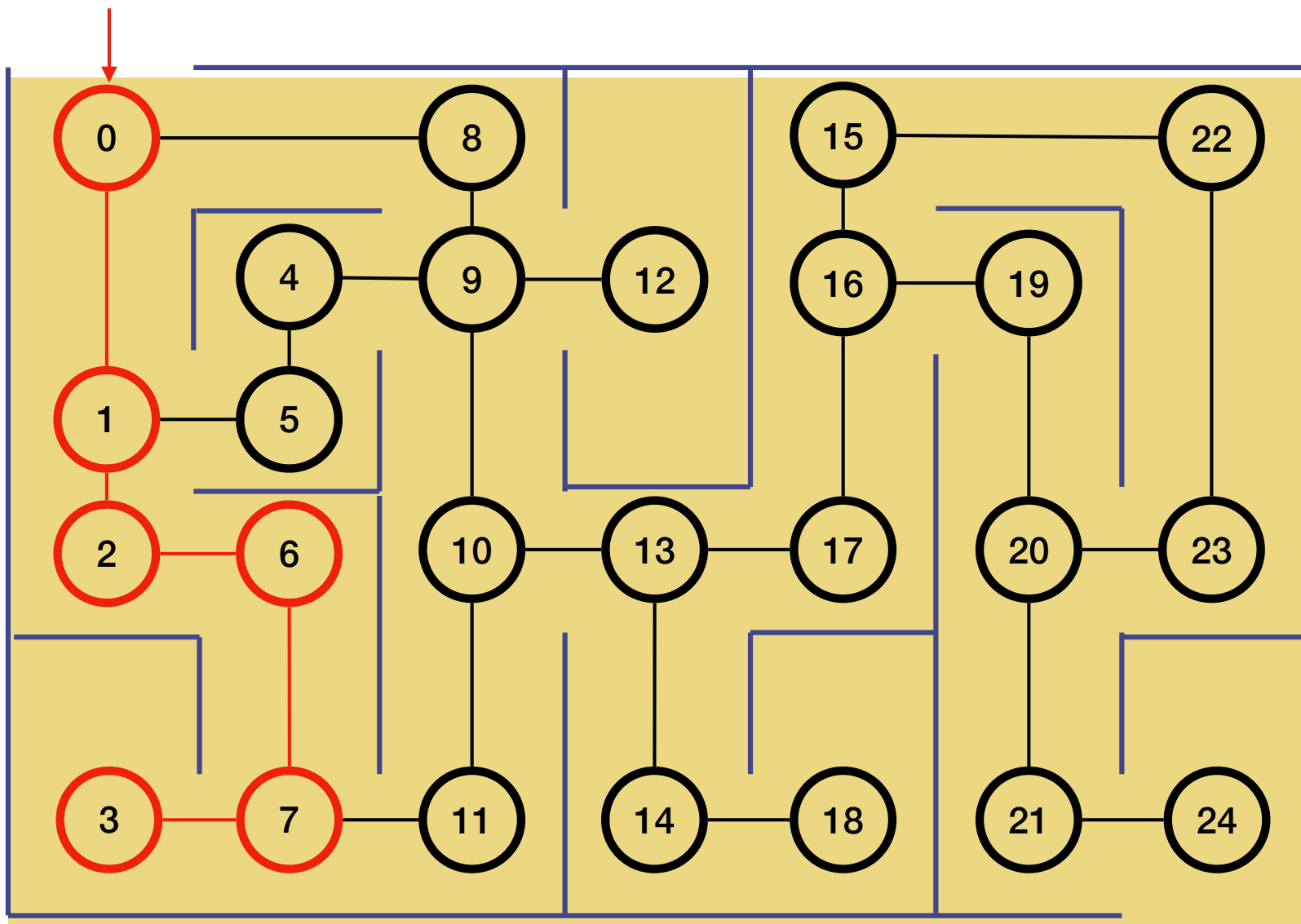




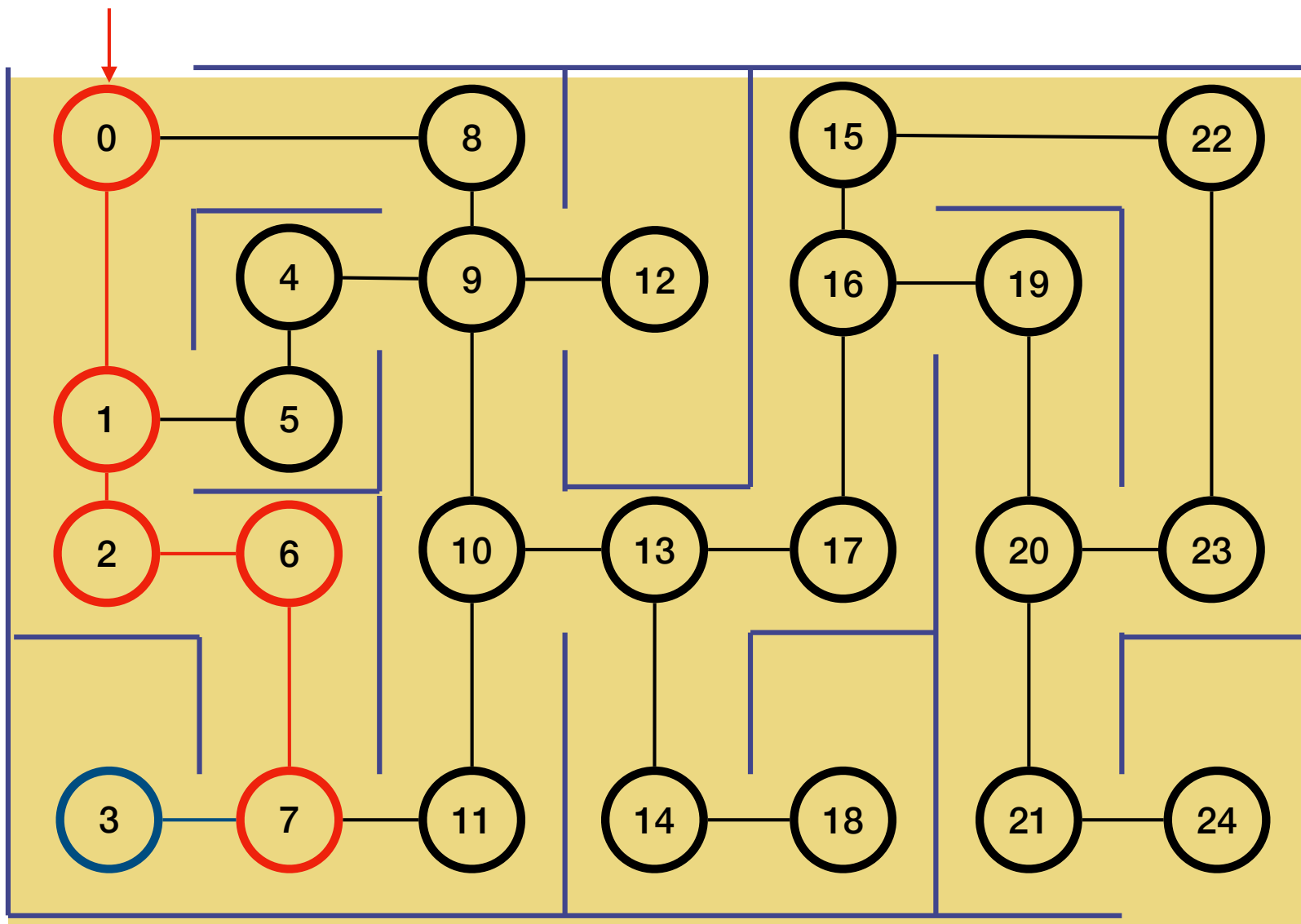
DFS



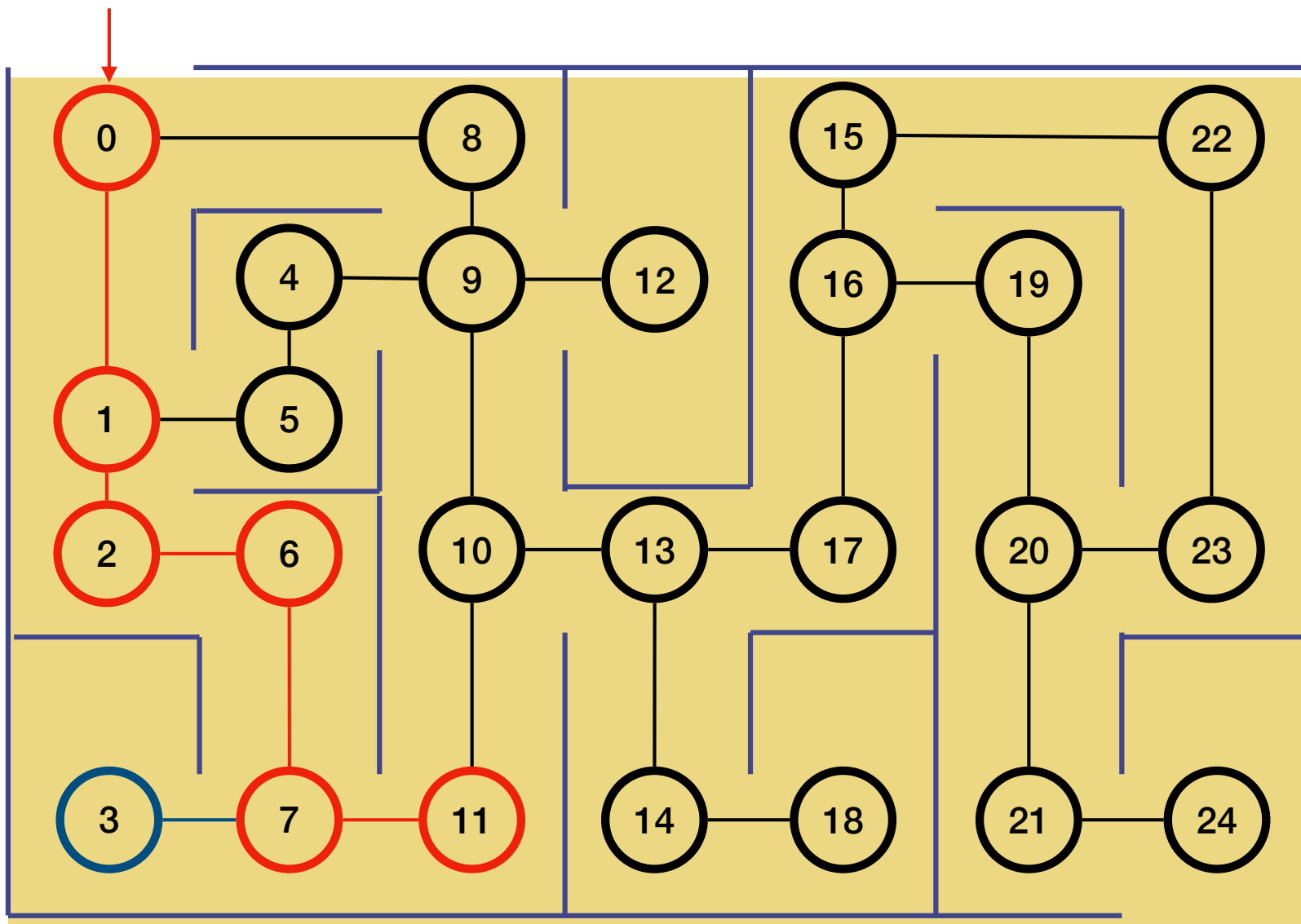
DFS



DFS

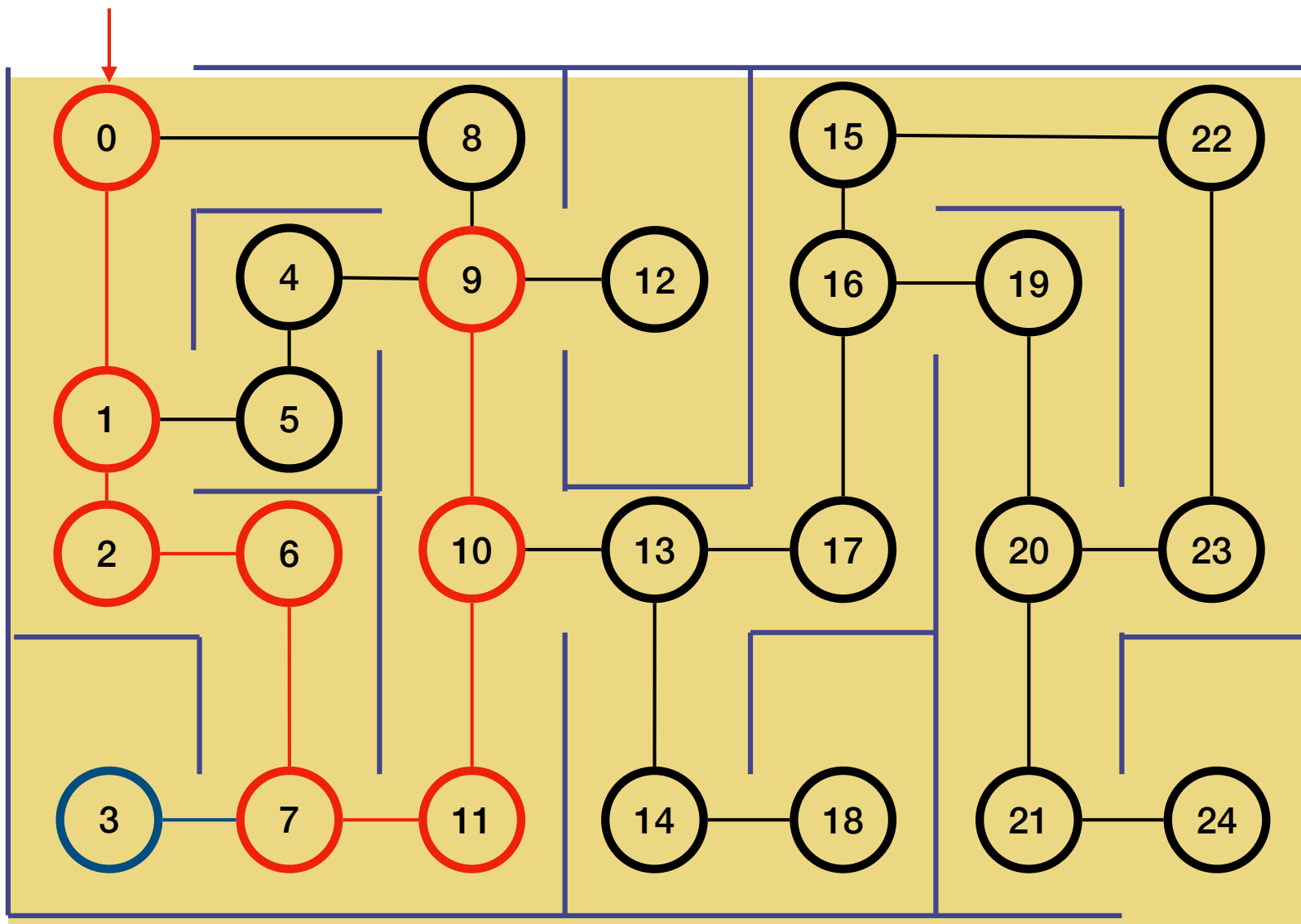


DFS

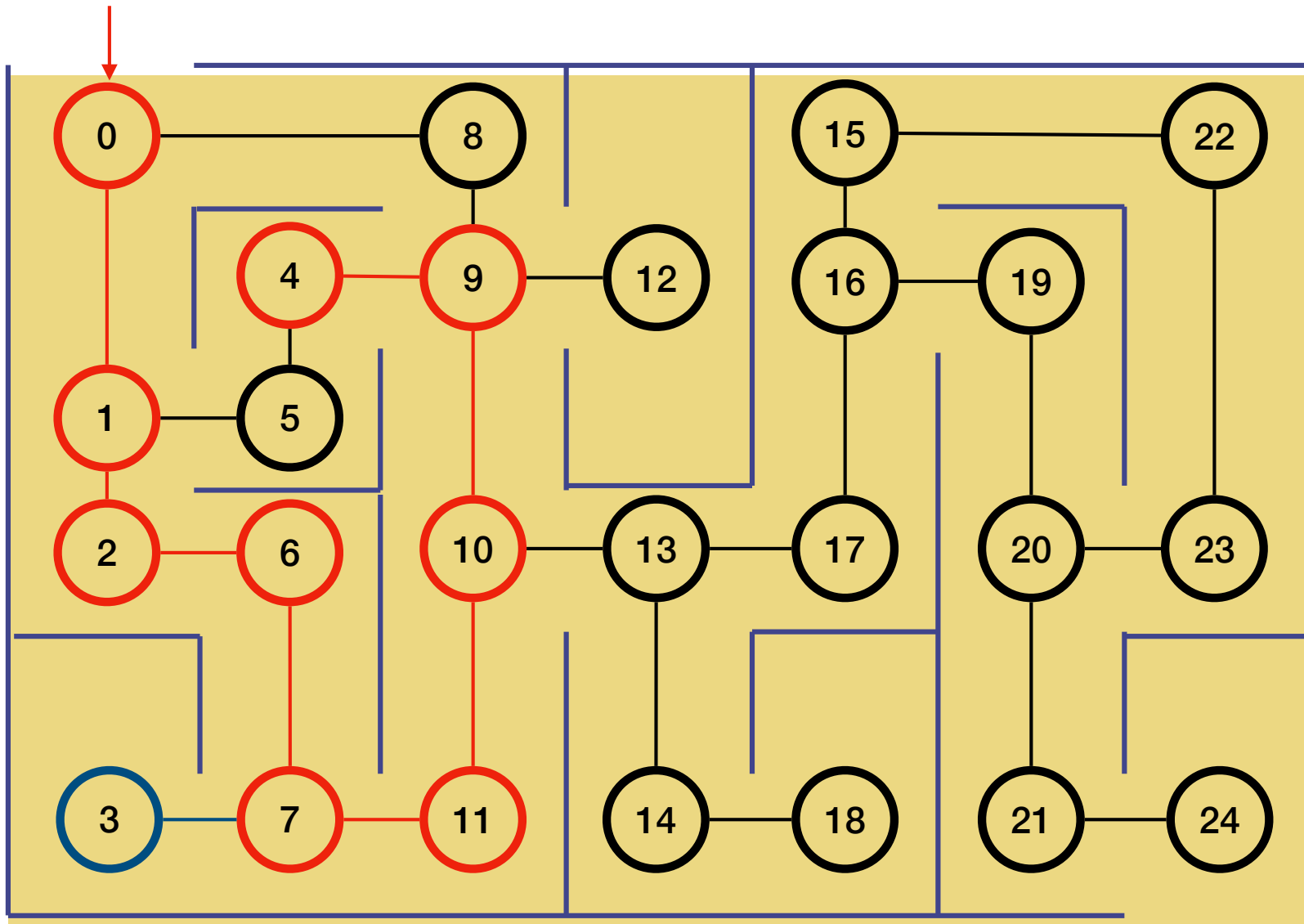




DFS

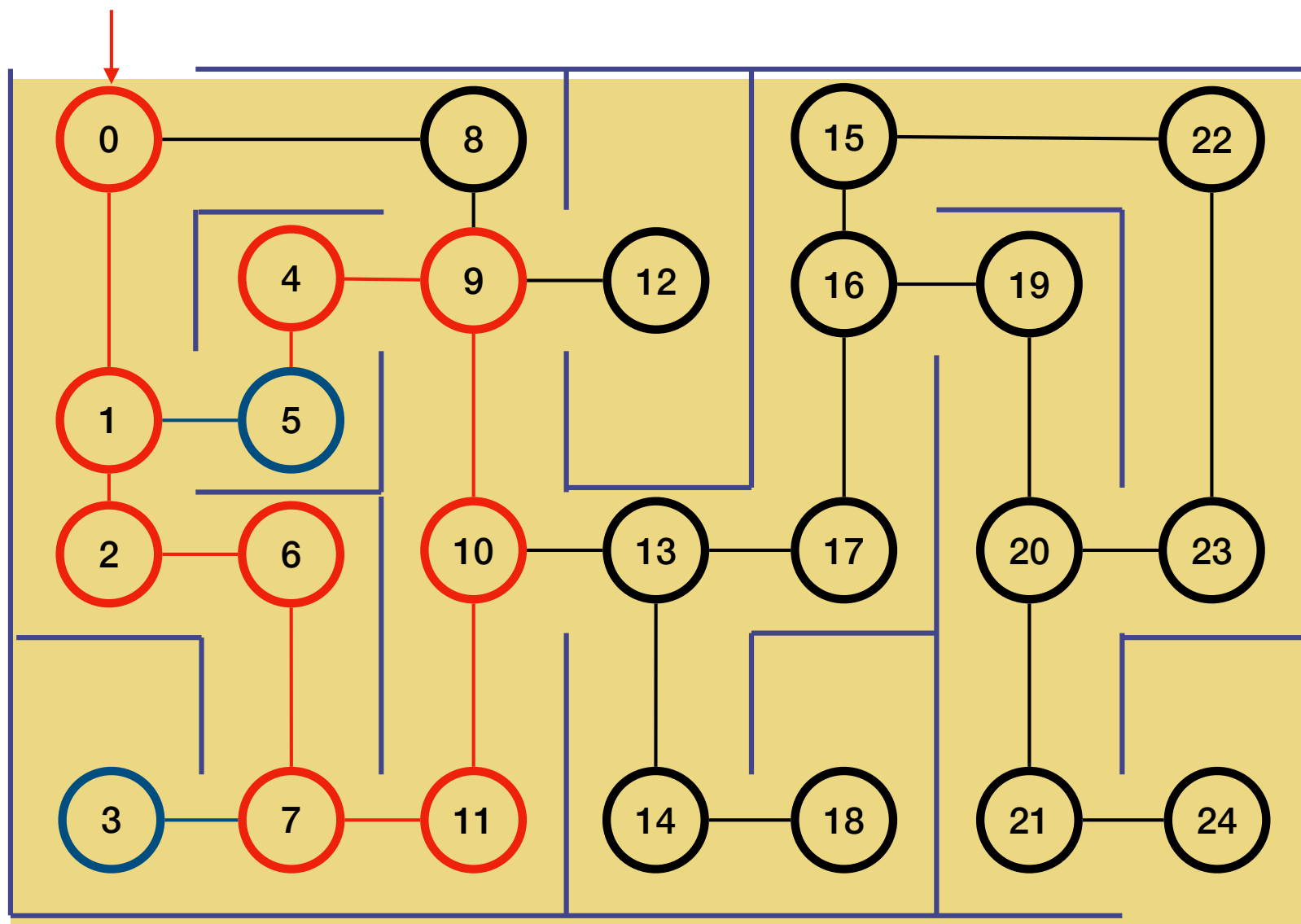


DFS

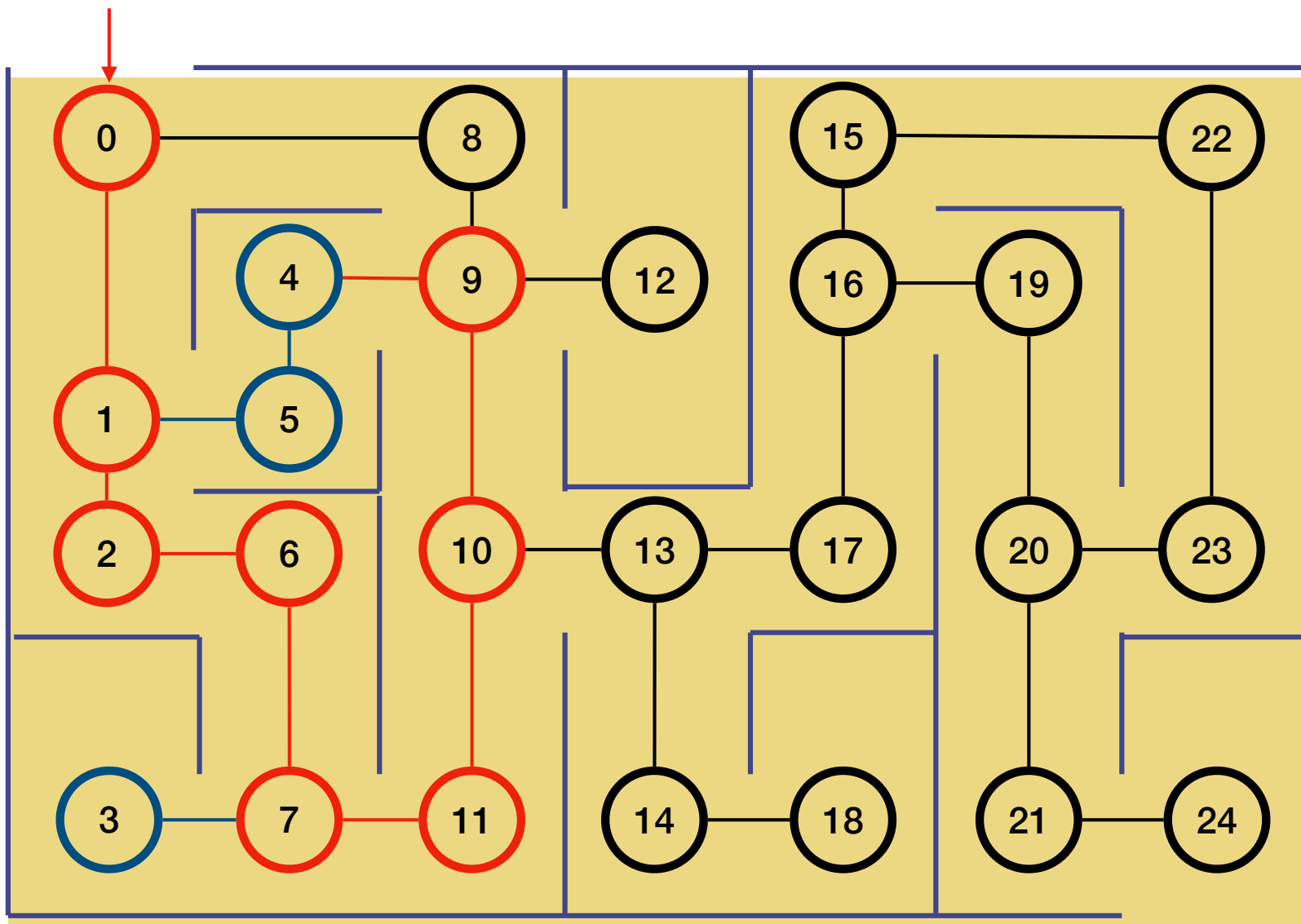




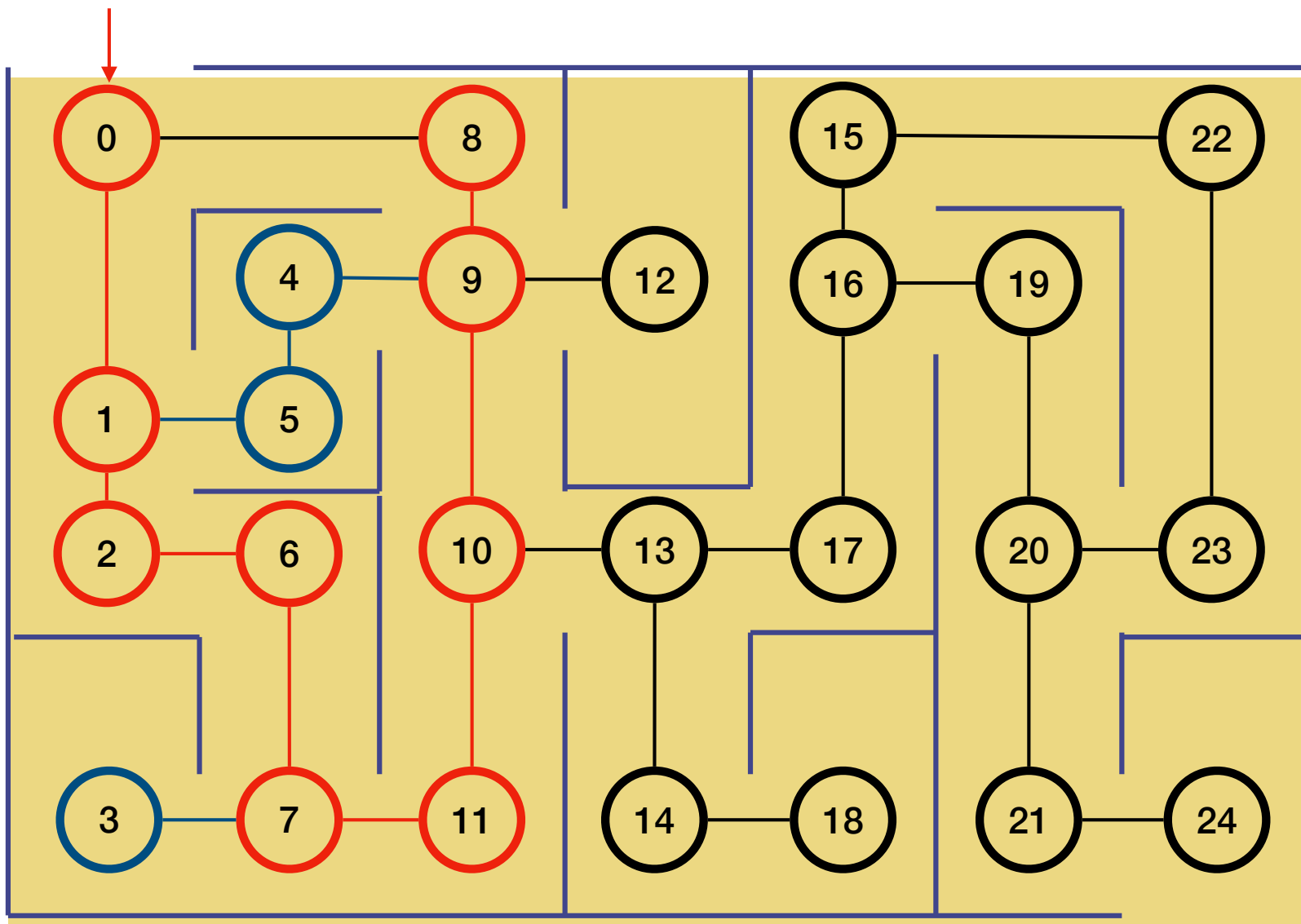
DFS



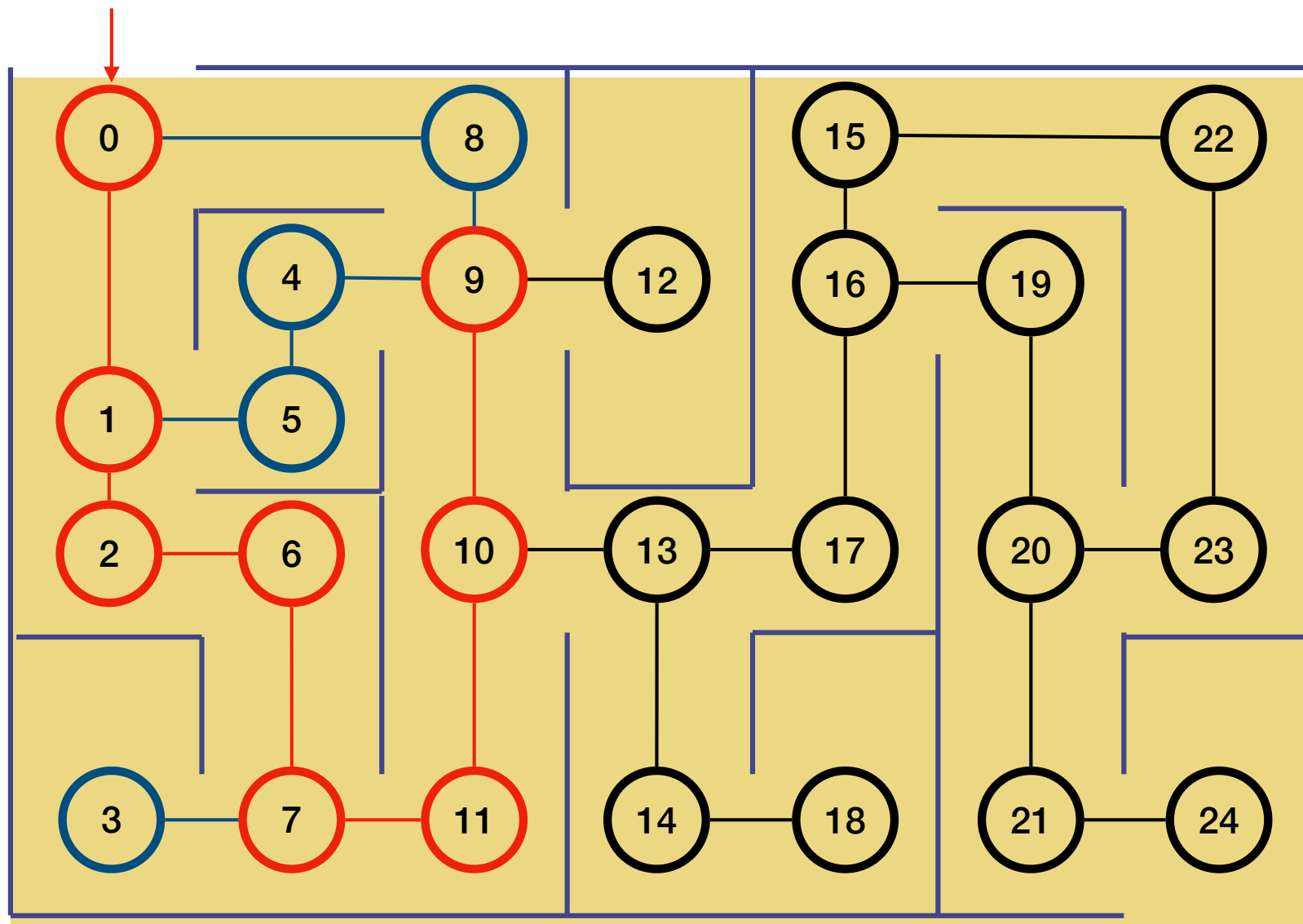
DFS



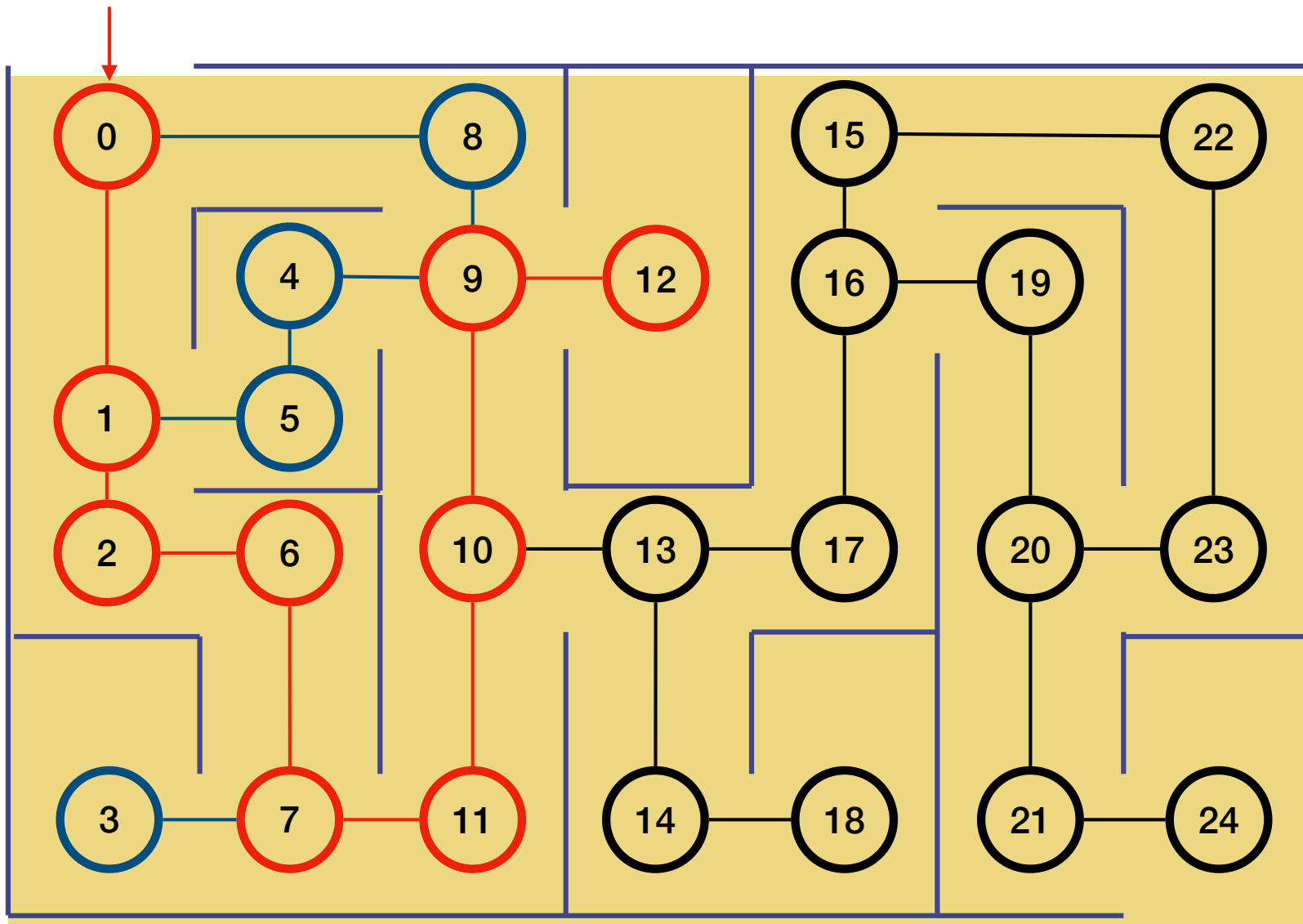
DFS



DFS

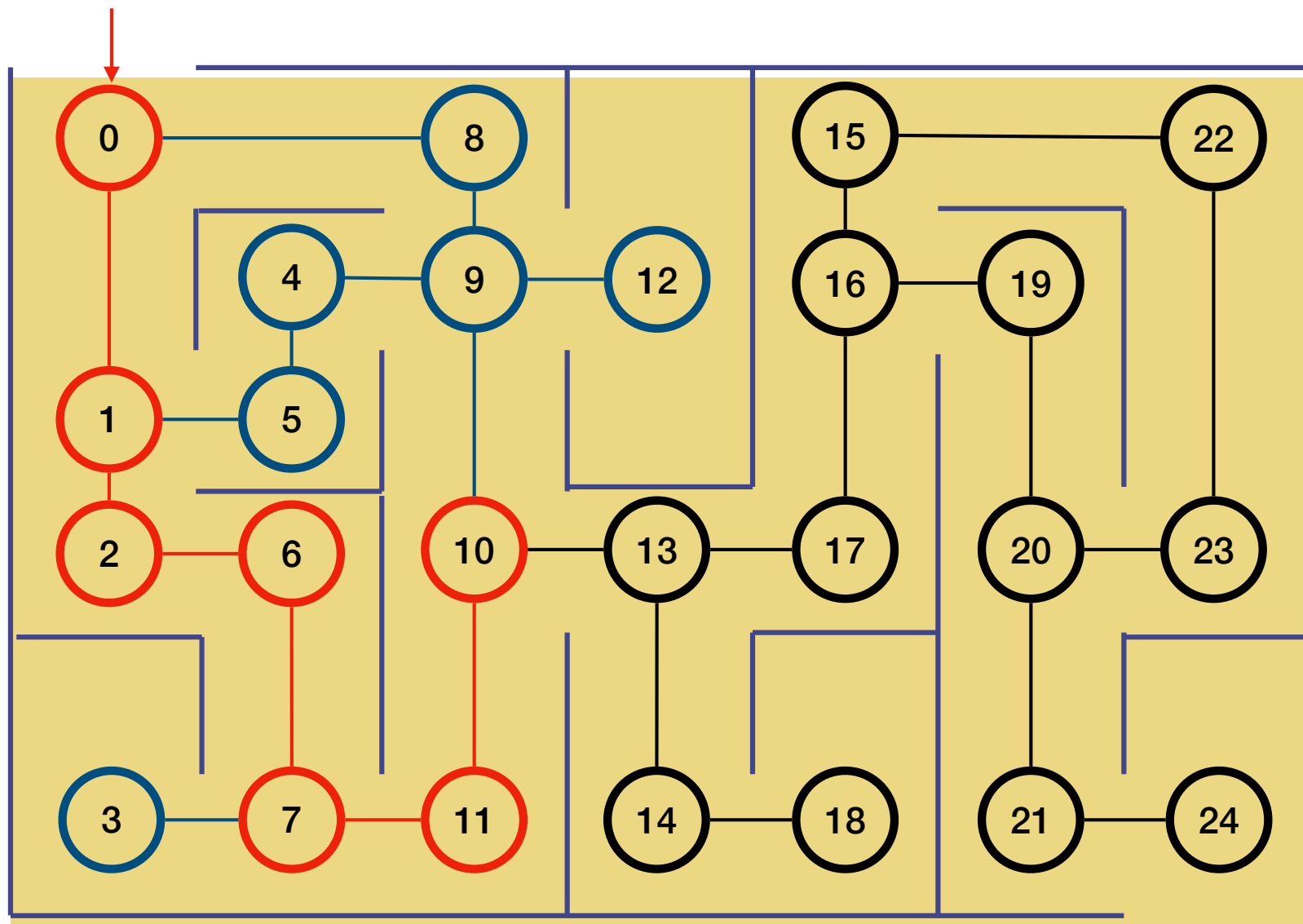


DFS

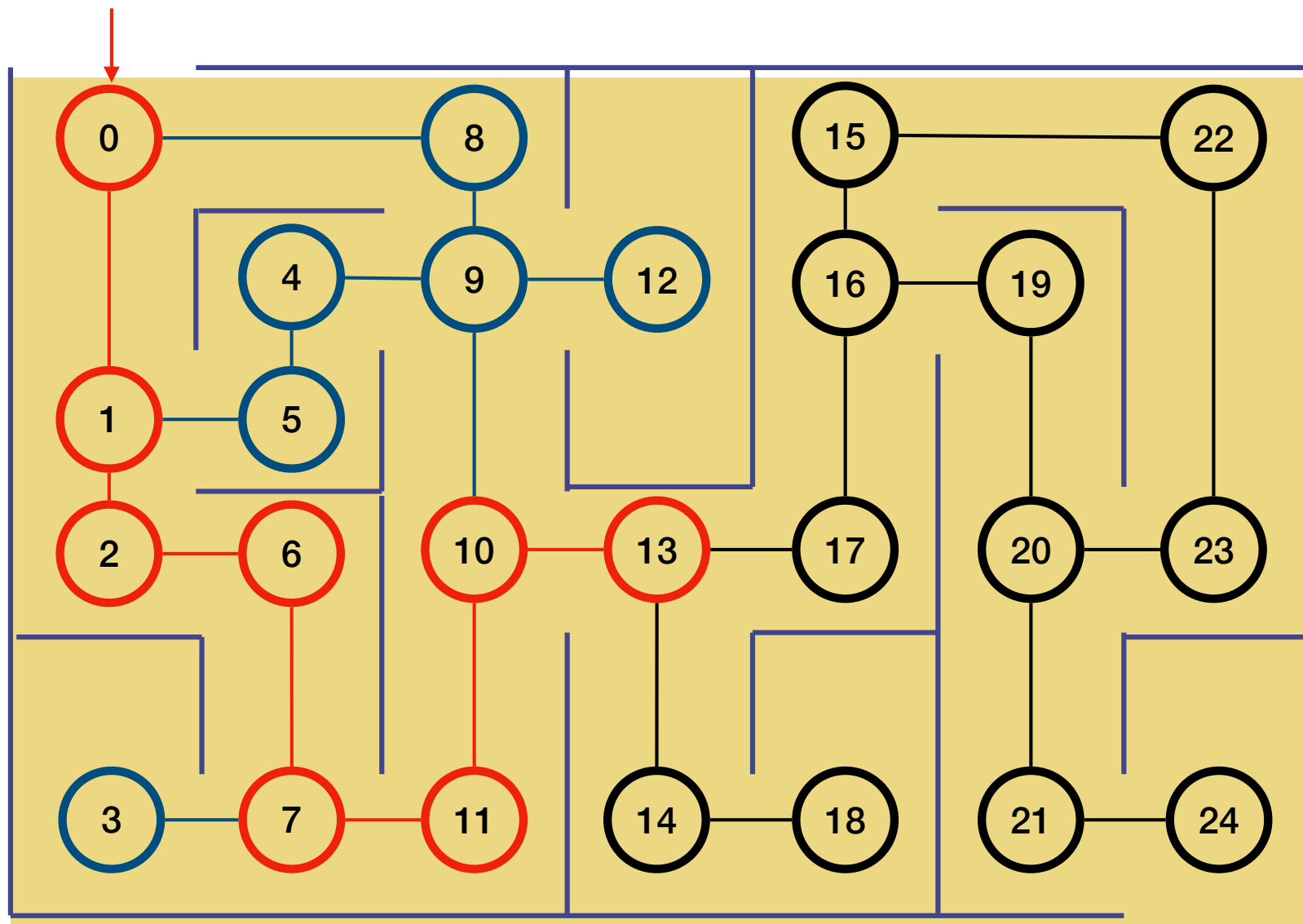




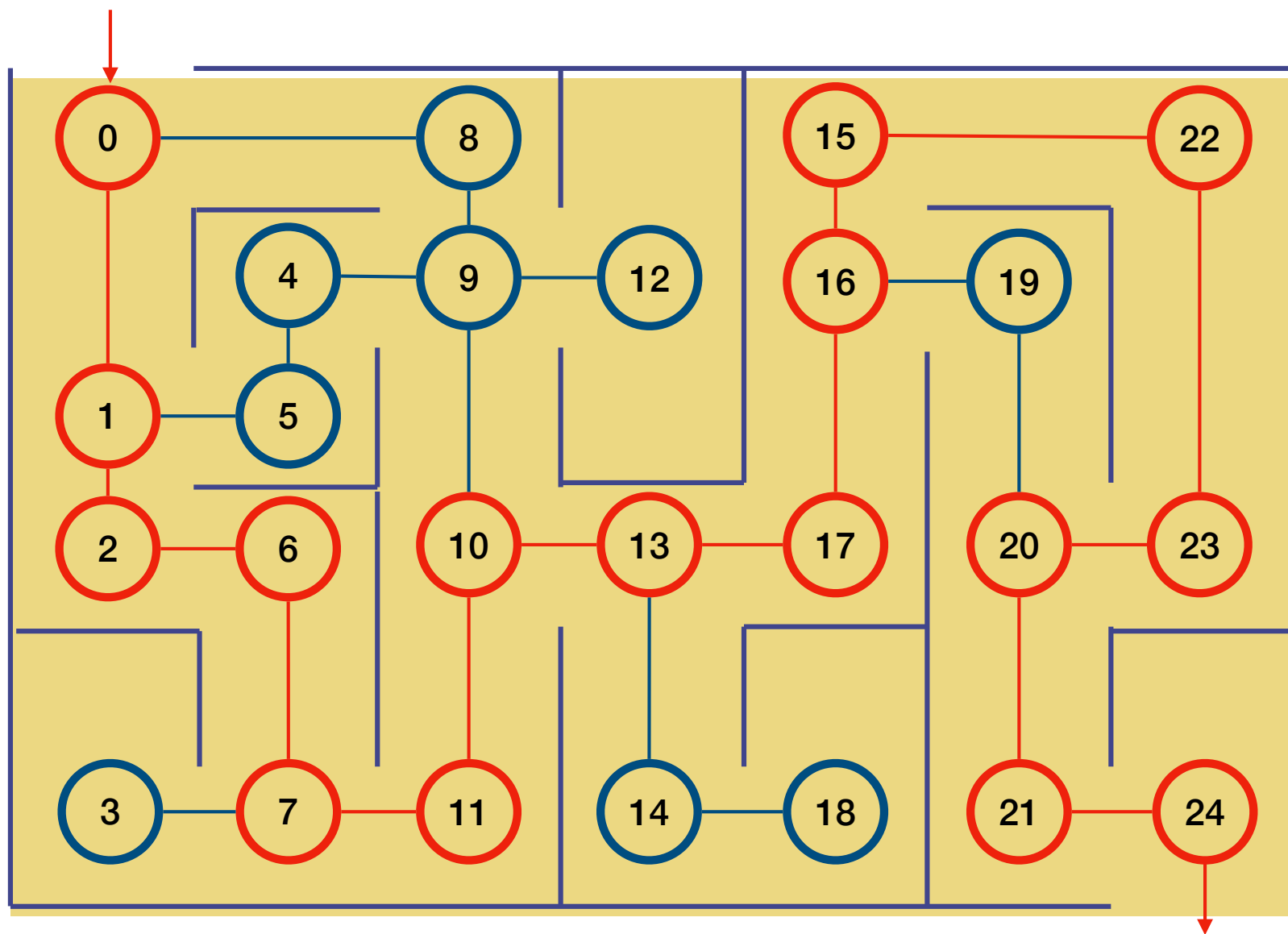
DFS



DFS



DFS



Les types d'arêtes parcourues par DFS

On utilise DFS pour analyser la structure d'un graphe.

La DFS identifie naturellement ce qu'on appelle l'arbre de recherche en profondeur (arbre DFS) enraciné au sommet de départ s .

Chaque fois qu'une arête $e = (u, v)$ est empruntée pour découvrir un nouveau sommet v , cette arête nous y menant est dite découverte (ou arête de l'arbre DFS).

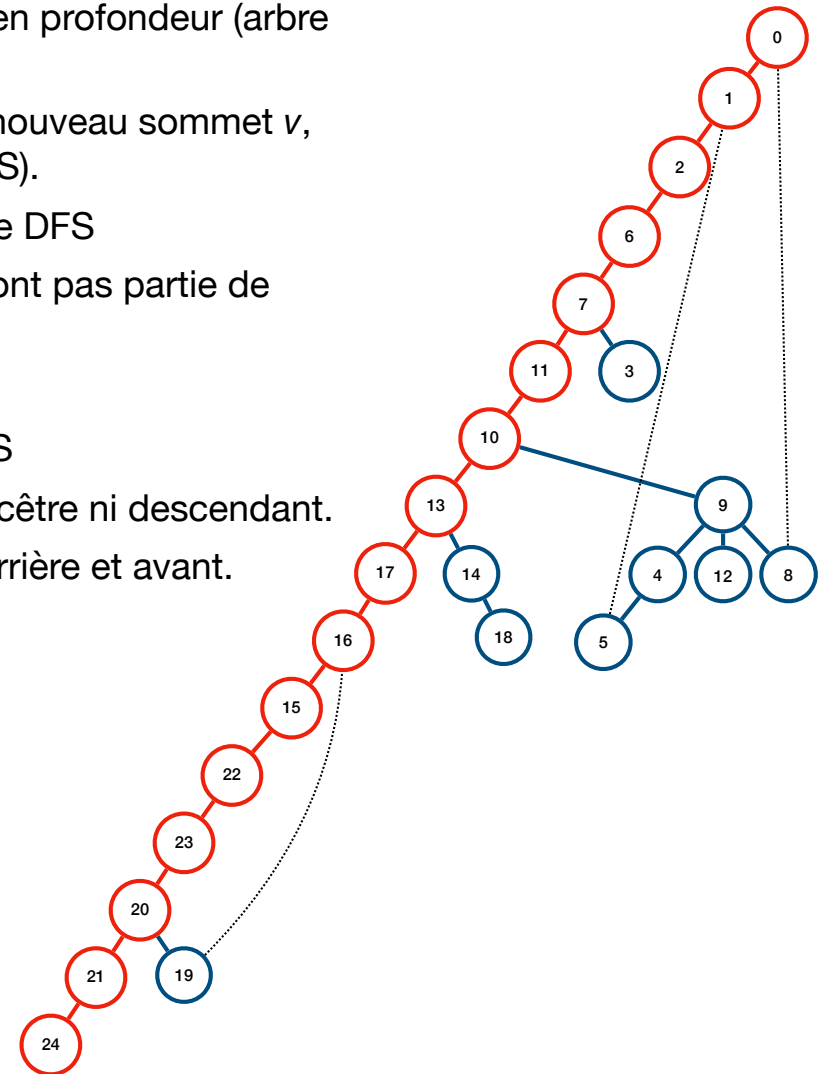
- arête **découverte** (ou **d'arbre**), relie deux sommets dans l'arbre DFS

Toutes les autres arêtes nous mènent à un sommet déjà visité et ne font pas partie de l'arbre DFS (non-DFS). Ces arêtes peuvent être de trois types :

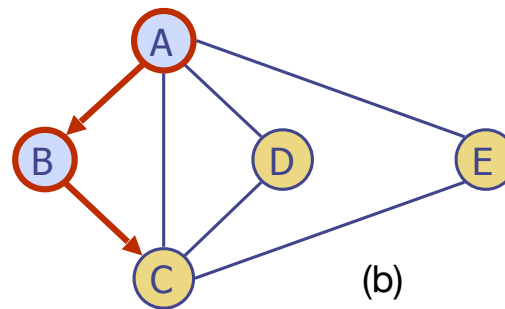
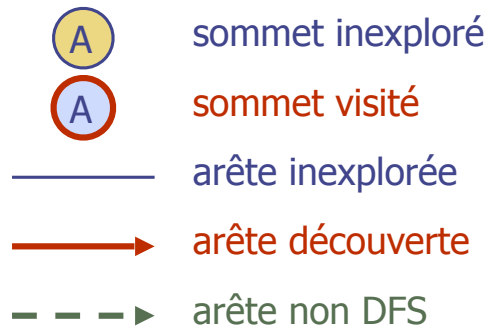
1. arête **arrière**, relie un sommet à un ancêtre de l'arbre DFS
2. arête **avant**, relie un sommet à un descendant dans l'arbre DFS
3. arête **transversales**, relie un sommet à un autre qui n'est ni ancêtre ni descendant.

👉 dans un graphe non-orienté, on ne distingue pas les arêtes arrière et avant.

👉 ***l'ordre dans lequel les arêtes sont parcourues importe.***

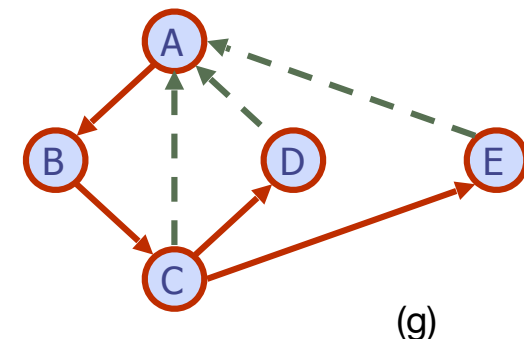
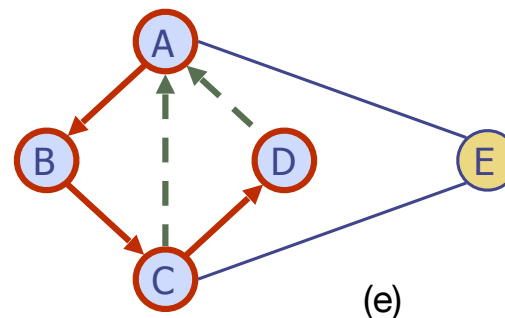
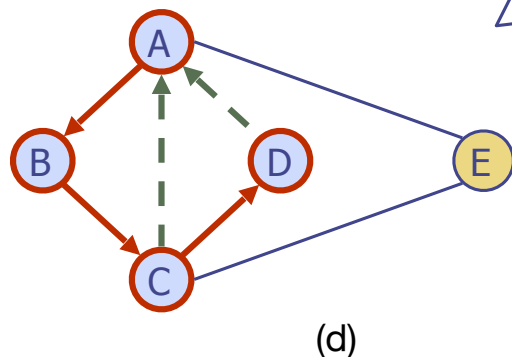
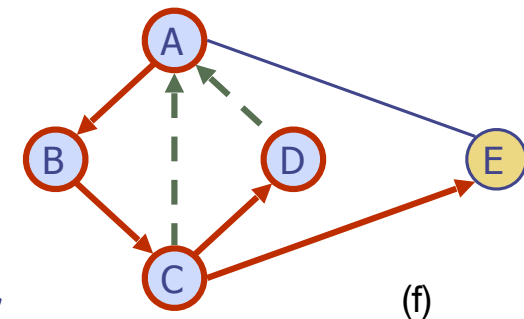
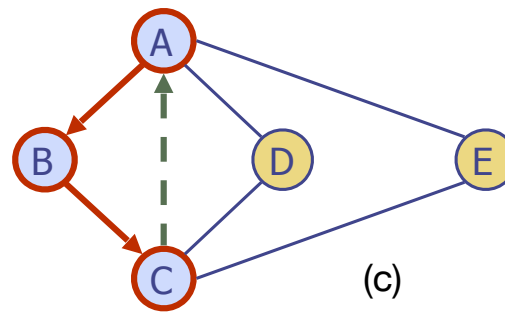
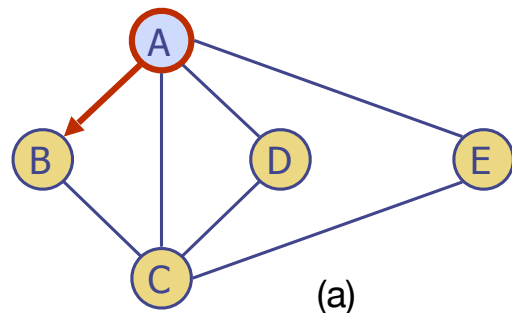


Exemple de DFS dans un graphe non-orienté

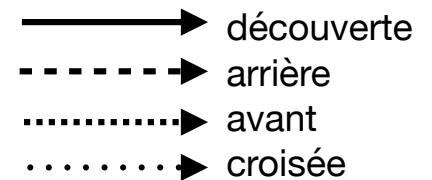
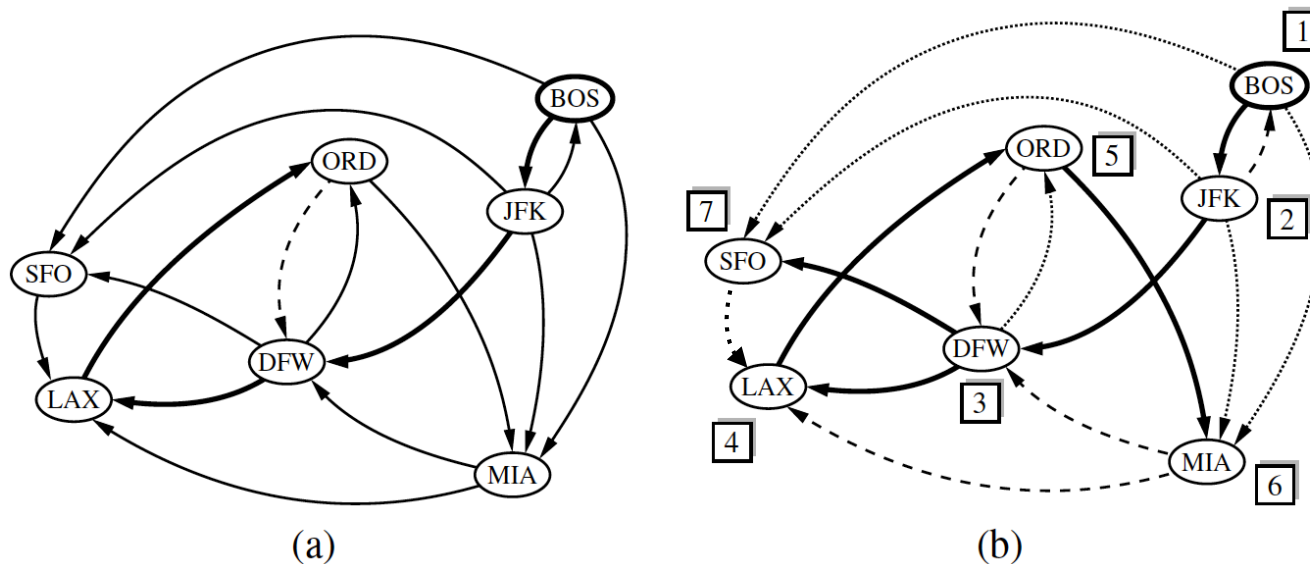


À partir du sommet A :

- (a) Après avoir parcouru l'arête AB. Le sommet B n'avait pas été parcouru, donc l'arête AB fait partie de l'arbre DFS.
- (b) Après avoir parcouru l'arête BC. C n'ayant pas été parcouru auparavant, l'arête BC fait partie de l'arbre DFS.
- (c) Le sommet A a déjà été parcouru. L'arête CA ne fait donc pas partie des arêtes découvertes.
- (defg) Et ainsi de suite...



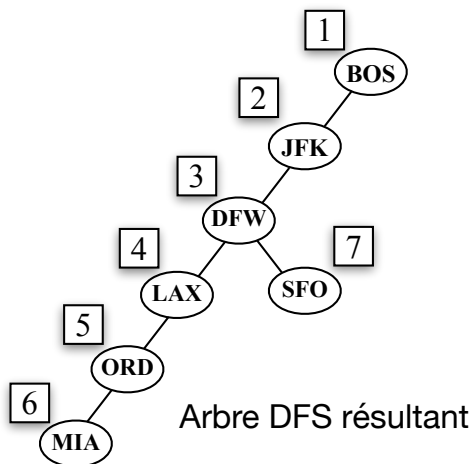
Exemple de DFS dans un graphe orienté



À partir du sommet BOS :

(a) **Étape intermédiaire.** Pour la première fois, une arête considérée conduit à un sommet visité (DFW);

(b) **DFS complétée.** L'ordre de visite des sommets est indiqué par une étiquette. L'arête (ORD, DFW) est une **arête arrière**, mais (DFW, ORD) est une **arête avant**. L'arête (SFO, LAX) est une **arête croisée**.



Propriétés de la DFS

Propriété *Arbre Couvrant DFS* : Soit G un graphe non-orienté. Si l'on exécute une DFS à partir d'un sommet s , alors les arêtes découvertes (arêtes d'arbre) forment un arbre couvrant de la composante connexe contenant s .
Justification:

Absence de cycle. Les arêtes d'arbre sont exactement les arêtes qui mènent à des sommets non encore visités.

Lorsqu'une arête (u, v) est ajoutée à l'arbre DFS, le sommet v n'a pas encore été visité. Il est donc impossible qu'un chemin relie déjà u à v , sinon v aurait déjà été marqué comme visité.

Ainsi, aucune arête d'arbre ne peut fermer un cycle. Les arêtes découvertes forment donc un sous-graphe acyclique.

Connexité. Chaque sommet visité (sauf s) est atteint à partir d'un sommet déjà visité par une arête d'arbre. Il existe donc un chemin, composé d'arêtes d'arbre, reliant chaque sommet visité à s . Le sous-graphe formé par ces arêtes est donc connexe.

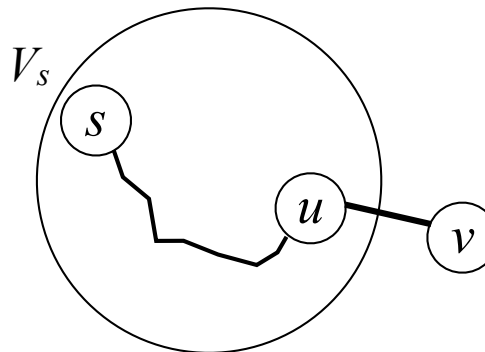
Caractère couvrant. Montrons que tous les sommets de la composante connexe contenant s , V_s , sont visités. Supposons par contradiction qu'il existe un sommet v de cette composante qui n'a pas été visité.

Alors :

- certains sommets ont été visités,
- d'autres ne l'ont pas été.

Comme G est connexe (dans cette composante), il existe une arête reliant un sommet visité u à un sommet non visité v . Mais lorsque l'algorithme traite u , il examine tous ses voisins. Il aurait donc visité v , contradiction.

Donc, tous les sommets de la composante contenant s sont visités.



Performances de la DFS

La DFS est une méthode efficace pour parcourir un graphe. Elle est appelée au plus une fois sur chaque sommet (puisqu'ils sont marqués lors de la première visite) et chaque arête est considérée au plus deux fois (pour un graphe non-orienté ; une fois de chaque extrémité), et au plus une fois dans un graphe orienté (à partir de son sommet d'origine).

Si on définit $n_s \leq n$ comme le nombre de sommets accessibles depuis un sommet s , et $m_s \leq m$ le nombre d'arêtes adjacentes à ces sommets, la DFS commençant à s exécute dans $O(n_s + m_s)$, si on a les conditions suivantes :

- L'opération **outgoingEdges**(v) est dans $O(\deg(v))$, et **opposite**(v, e) est dans $O(1)$, réalisés avec une liste d'adjacence, par exemple, mais pas avec une matrice d'adjacence.
- "Marquer" un sommet ou une arête et tester si un sommet ou une arête ont été explorés est dans $O(1)$.

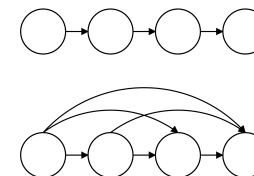
Ainsi, on peut résoudre un certain nombre de problèmes intéressants de manière efficace :

Dans un **graphe G non-orienté de n sommets et m arêtes**, la DFS résout les problèmes suivants dans **$O(n + m)$** :

- Calculer un chemin entre deux sommets de G , s'il en existe un
- Tester si G est connexe
- Calculer un arbre couvrant de G , si G est connexe
- Calculer les composantes connexes de G
- Trouver un cycle dans G ou déterminer que G ne contient pas de cycle

Dans un **graphe G orienté avec n sommets et m arêtes**, la DFS résout les problèmes suivants dans **$O(n + m)$** :

- Calculer un chemin dirigé entre deux sommets de G , s'il en existe un
- Calculer l'ensemble des sommets de G qui sont accessibles depuis un sommet donné s
- Tester si G est fortement connexe
- Trouver un cycle dirigé dans G , ou déterminer que G est acyclique
- Calculer la fermeture transitive de G (à voir en IFT2125 ?)



Les arcs de la fermeture transitive sont les couples de sommets entre lesquels il existe un chemin

```

public static <V,E> void DFS( Graph<V,E> g, Vertex<V> u,
                             Set<Vertex<V>> known,
                             Map<Vertex<V>,Edge<E>> forest ) {
    known.add( u );
    for( Edge<E> e : g.outgoingEdges( u ) ) {
        Vertex<V> v = g.opposite( u, e );
        if( !known.contains( v ) ) {
            forest.put( v, e );
            DFS( g, v, known, forest );
        }
    }
}

```

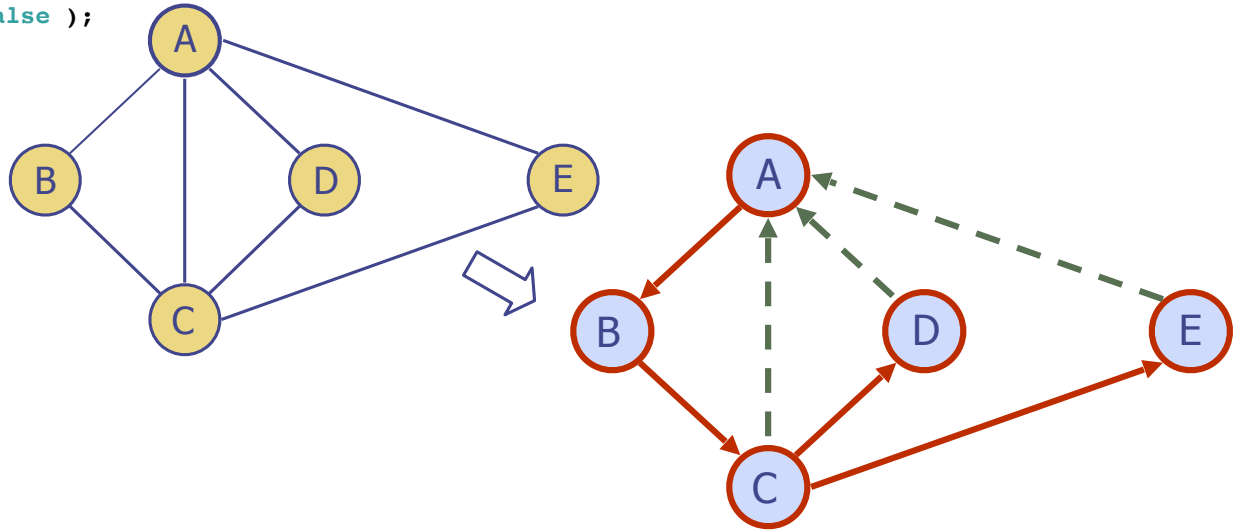
```
Graph<String,String> G = new AdjacencyMapGraph<>( false );
```

```
Vertex<String> aa = G.insertVertex( "A" );
Vertex<String> bb = G.insertVertex( "B" );
Vertex<String> cc = G.insertVertex( "C" );
Vertex<String> dd = G.insertVertex( "D" );
Vertex<String> ee = G.insertVertex( "E" );
```

```
Edge<String> ab = G.insertEdge( aa, bb, "AB" );
Edge<String> ac = G.insertEdge( aa, cc, "AC" );
Edge<String> ad = G.insertEdge( aa, dd, "AD" );
Edge<String> ae = G.insertEdge( aa, ee, "AE" );
Edge<String> bc = G.insertEdge( bb, cc, "BC" );
Edge<String> cd = G.insertEdge( cc, dd, "CD" );
Edge<String> ce = G.insertEdge( cc, ee, "CE" );
System.out.println( G );
```

```
Vertex A
  4 adjacencies: (D, AD) (C, AC) (E, AE) (B, AB)
Vertex B
  2 adjacencies: (A, AB) (C, BC)
Vertex C
  4 adjacencies: (D, CD) (A, AC) (E, CE) (B, BC)
Vertex D
  2 adjacencies: (C, CD) (A, AD)
Vertex E
  2 adjacencies: (C, CE) (A, AE)
```

```
Set<Vertex<String>> discovered = new HashSet<>();
Map<Vertex<String>,Edge<String>> forest = new HashMap<>();
AdjacencyMapGraph.DFS( G, aa, discovered, forest );
System.out.println( "Path between vertices " + aa.getElement() + " and " + ee.getElement() + ":" );
for( Edge<String> e : AdjacencyMapGraph.constructPath( G, aa, ee, forest ) )
  System.out.println( e.getElement() );
```



Path between vertices A and E:

AB

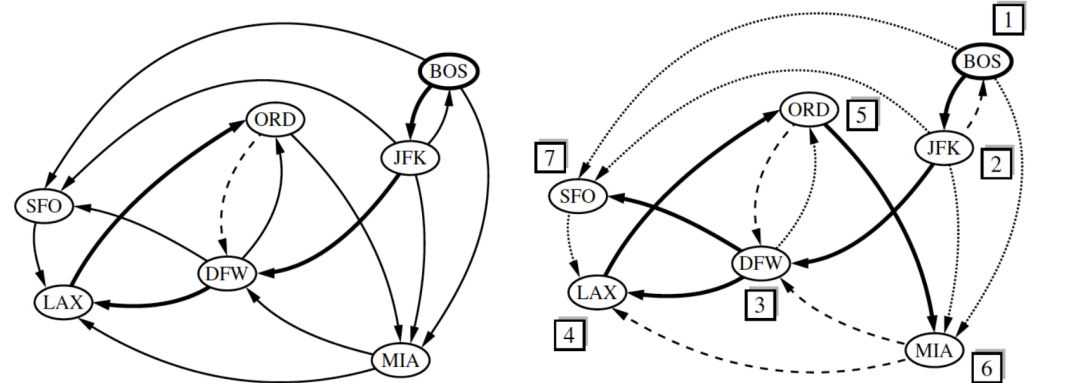
BC

CE

```
Graph<String,String> gl48 = new AdjacencyMapGraph<>( true );
```

```
Vertex<String> bos = gl48.insertVertex( "BOS" );
Vertex<String> jfk = gl48.insertVertex( "JFK" );
Vertex<String> mia = gl48.insertVertex( "MIA" );
Vertex<String> chi = gl48.insertVertex( "ORD" );
Vertex<String> dfw = gl48.insertVertex( "DFW" );
Vertex<String> sfo = gl48.insertVertex( "SFO" );
Vertex<String> lax = gl48.insertVertex( "LAX" );
```

```
Edge<String> bosjfk = gl48.insertEdge( bos, jfk, "BOS->JFK" );
Edge<String> bosmia = gl48.insertEdge( bos, mia, "BOS->MIA" );
Edge<String> bossfo = gl48.insertEdge( bos, sfo, "BOS->SFO" );
Edge<String> jfkbos = gl48.insertEdge( jfk, bos, "JFK->BOS" );
Edge<String> jfkdfw = gl48.insertEdge( jfk, dfw, "JFK->DFW" );
Edge<String> jfkmia = gl48.insertEdge( jfk, mia, "JFK->MIA" );
Edge<String> jfksfo = gl48.insertEdge( jfk, sfo, "JFK->SFO" );
Edge<String> chimia = gl48.insertEdge( chi, mia, "ORD->MIA" );
Edge<String> chidfw = gl48.insertEdge( chi, dfw, "ORD->DFW" );
Edge<String> sfolax = gl48.insertEdge( sfo, lax, "SFO->LAX" );
Edge<String> laxchi = gl48.insertEdge( lax, chi, "LAX->ORD" );
Edge<String> dfwlax = gl48.insertEdge( dfw, lax, "DFW->LAX" );
Edge<String> dfwsfo = gl48.insertEdge( dfw, sfo, "DFW->SFO" );
Edge<String> dfwchi = gl48.insertEdge( dfw, chi, "DFW->ORD" );
Edge<String> miadfw = gl48.insertEdge( mia, dfw, "MIA->DFW" );
Edge<String> mialax = gl48.insertEdge( mia, lax, "MIA->LAX" );
System.out.println( gl48 );
```



```
Vertex BOS
[outgoing] 3 adjacencies: (MIA, BOS->MIA) (JFK, BOS->JFK) (SFO, BOS->SFO)
[incoming] 1 adjacencies: (JFK, JFK->BOS)
Vertex JFK
[outgoing] 4 adjacencies: (MIA, JFK->MIA) (SFO, JFK->SFO) (BOS, JFK->BOS) (DFW, JFK->DFW)
[incoming] 1 adjacencies: (BOS, BOS->JFK)
Vertex MIA
[outgoing] 2 adjacencies: (LAX, MIA->LAX) (DFW, MIA->DFW)
[incoming] 3 adjacencies: (ORD, ORD->MIA) (JFK, JFK->MIA) (BOS, BOS->MIA)
Vertex ORD
[outgoing] 2 adjacencies: (MIA, ORD->MIA) (DFW, ORD->DFW)
[incoming] 2 adjacencies: (DFW, DFW->ORD) (LAX, LAX->ORD)
Vertex DFW
[outgoing] 3 adjacencies: (SFO, DFW->SFO) (LAX, DFW->LAX) (ORD, DFW->ORD)
[incoming] 3 adjacencies: (MIA, MIA->DFW) (JFK, JFK->DFW) (ORD, ORD->DFW)
Vertex SFO
[outgoing] 1 adjacencies: (LAX, SFO->LAX)
[incoming] 3 adjacencies: (BOS, BOS->SFO) (DFW, DFW->SFO) (JFK, JFK->SFO)
Vertex LAX
[outgoing] 1 adjacencies: (ORD, LAX->ORD)
[incoming] 3 adjacencies: (SFO, SFO->LAX) (DFW, DFW->LAX) (MIA, MIA->LAX)
```

Path between vertices BOS and SFO:
BOS->JFK
JFK->DFW
DFW->SFO

```
discovered = new HashSet<>();
forest = new ProbeHashMap<>();
AdjacencyMapGraph.DFS( gl48, bos, discovered, forest );
System.out.println( "Path between vertices " + bos.getElement() + " and " + sfo.getElement() + ":" );
for( Edge<String> e : AdjacencyMapGraph.constructPath( gl48, bos, sfo, forest ) )
    System.out.println( e.getElement() );
```

Trouver un chemin entre deux sommets

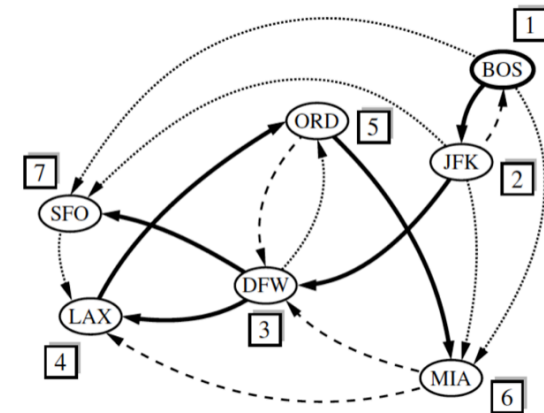
La DFS identifie un chemin (dirigé) entre les sommets u et v , si v est accessible de u . Ce chemin peut être reconstruit à partir des informations enregistrées dans le dictionnaire des arêtes découvertes. La fonction **constructPath**($g, u, v, discovered$) produit une liste ordonnée des sommets sur le chemin entre u et v .

Pour reconstruire le chemin, on commence à la destination.

Tant que le sommet origine n'est pas identifié, on remonte les arêtes en prenant le sommet opposé qu'on ajoute au chemin. Et on recommence à partir de ce dernier sommet.

Une fois que nous avons tracé le chemin de v à u , il suffit de l'inverser pour obtenir le chemin de u à v .

Ce processus prend un temps proportionnel à la longueur du chemin, donc dans $O(n)$ en pire cas, en plus du temps passé dans la DFS pour déterminer les arêtes découvertes.



```
public static <V,E> PositionalList<Edge<E>> constructPath( Graph<V,E> g,
                                                         Vertex<V> u,
                                                         Vertex<V> v,
                                                         Map<Vertex<V>,Edge<E>> forest ) {
    PositionalList<Edge<E>> path = new LinkedPositionalList<>();
    if( forest.get( v ) != null ) {
        Vertex<V> walk = v;
        while( walk != u ) {
            Edge<E> edge = forest.get( walk );
            path.addFirst( edge );
            walk = g.opposite( walk, edge );
        }
    }
    return path;
}
```

Chemin entre BOS et ORD :
 BOS
 JFK
 DFW
 LAX
 ORD

La recherche en largeur (BFS)

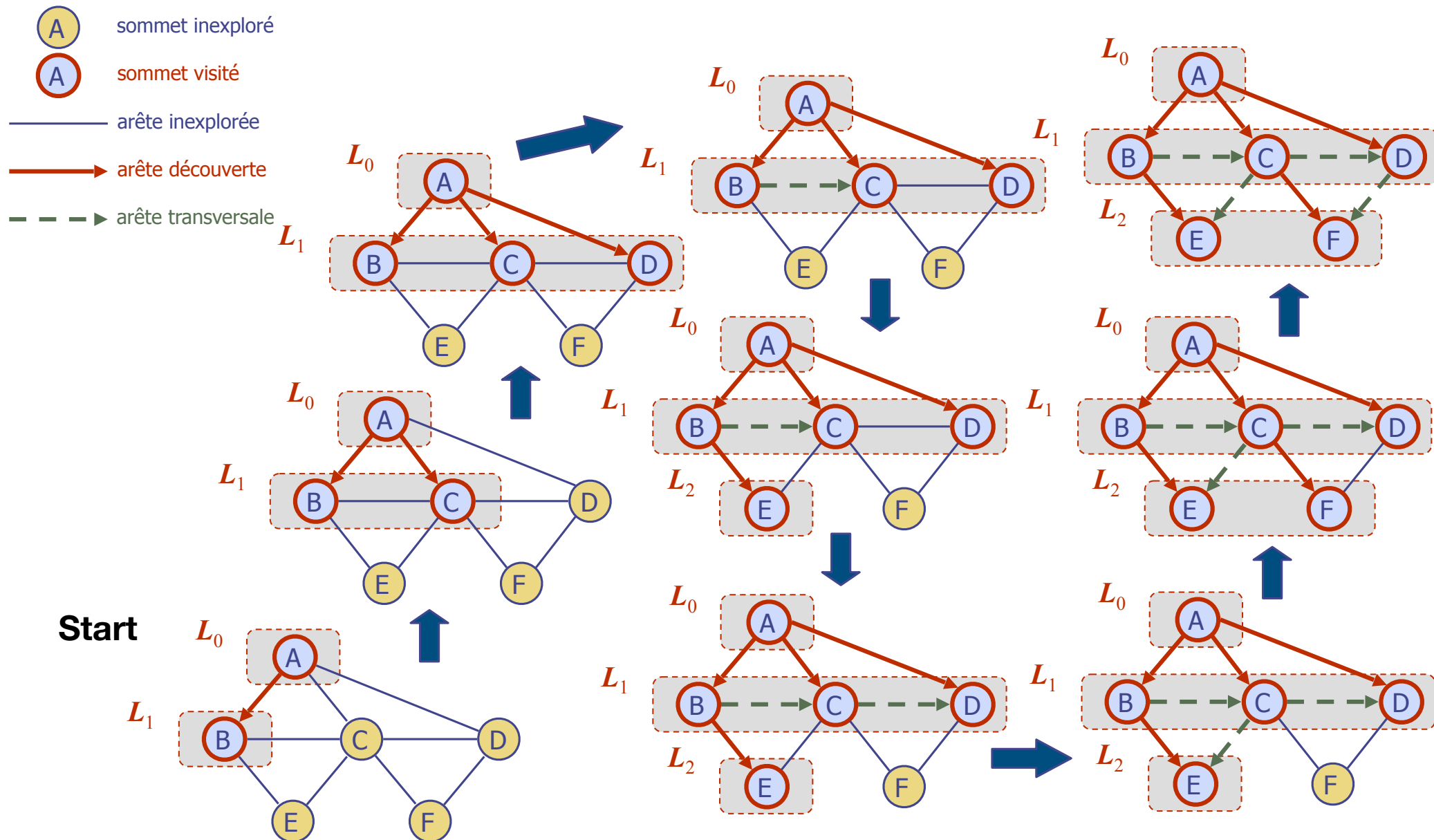
Un autre algorithme pour traverser une composante connexe d'un graphe est la recherche en largeur (BFS : Breadth-First-Search).

L'algorithme BFS est proche de l'envoi dans tous les sens de nombreux explorateurs qui traversent collectivement un graphe de manière coordonnée.

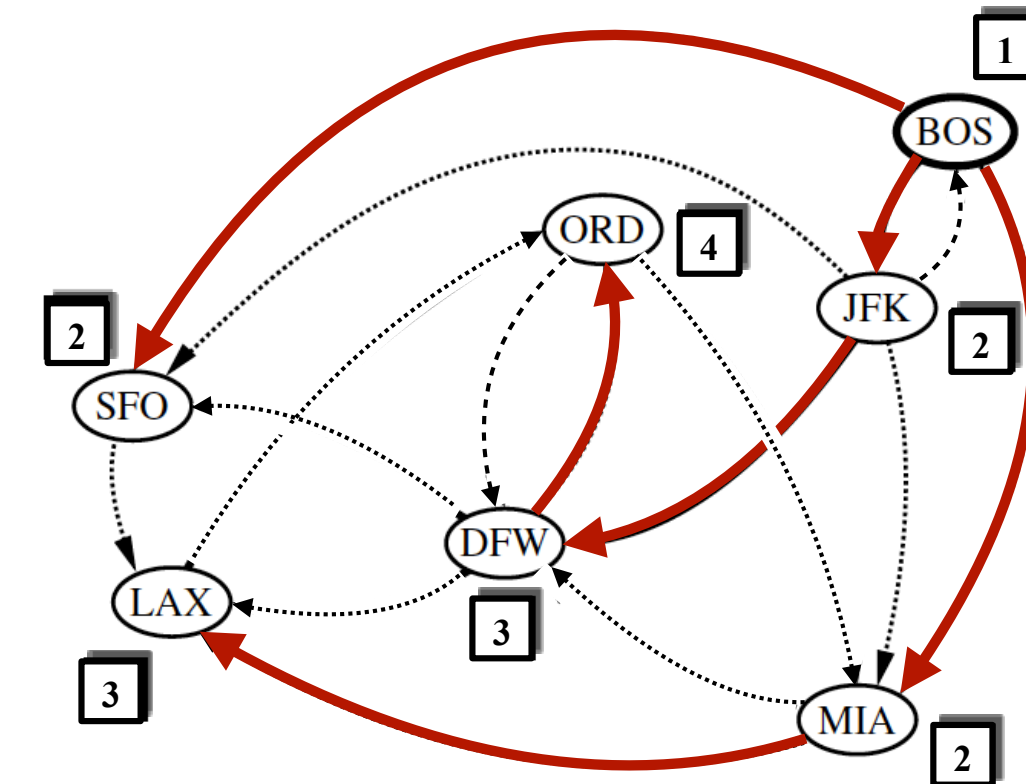
La BFS procède en rondes et subdivise les sommets en niveaux. La BFS démarre au sommet s , qui est au niveau 0. Au premier tour, on peint "visit  " tous les sommets adjacents au sommet s . Ces sommets sont    un pas du d  but et sont plac  s dans le niveau 1. Dans le deuxi  me tour, on permet    tous les explorateurs d'aller    deux pas du sommet de d  part. Ces nouveaux sommets, qui sont adjacents aux sommets de niveau 1 et qui n'ont pas   t   visit  s au niveau 1, sont plac  s dans le niveau 2 et marqu  s "visit  ". Ce processus se poursuit de mani  re similaire et se termine quand aucun nouveau sommet n'est trouv   pour cr  er un nouveau niveau.

```
public static <V,E> void BFS( Graph<V,E> g,
                             Vertex<V> s,
                             Set<Vertex<V>> known,
                             Map<Vertex<V>,Edge<E>> forest ) {
    PositionalList<Vertex<V>> level = new LinkedPositionalList<>();
    known.add( s );
    level.addLast( s );
    while( !level.isEmpty() ) {
        PositionalList<Vertex<V>> nextLevel = new LinkedPositionalList<>();
        for( Vertex<V> u : level )
            for( Edge<E> e : g.outgoingEdges( u ) ) {
                Vertex<V> v = g.opposite( u, e );
                if( !known.contains( v ) ) {
                    known.add( v );
                    forest.put( v, e );
                    nextLevel.addLast( v );
                }
            }
        level = nextLevel;
    }
}
```

Exemple de BFS dans un graphe non-orienté



Exemple de BFS dans un digraphe

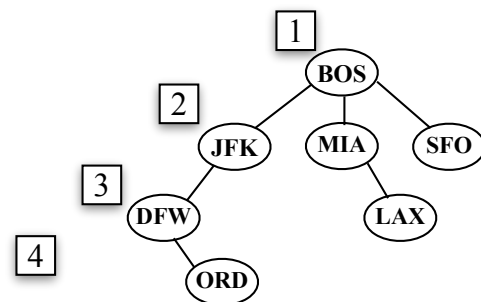


Les niveaux de la BFS sont indiqués par les chiffres dans les boîtes

L'arbre BFS est indiqué par les arêtes rouges

Path between vertices BOS and SFO:
BOS→SFO

BFS de BOS :
ORD DFW→ORD
DFW JFK→DFW
JFK BOS→JFK
MIA BOS→MIA
LAX MIA→LAX
SFO BOS→SFO



Arbre BFS résultant

—→ découverte (d'arbre)
- - -> arrière
.....> croisée

Classification des arêtes pour BFS

Graphe non orienté

Dans un graphe non orienté, lors d'une BFS :

- Les arêtes de l'arbre BFS sont appelées **arêtes d'arbre**.
- Toutes les autres arêtes sont des **arêtes croisées (transversales)**.

Il n'y a ni arêtes arrière ni arêtes avant comme en DFS.

Graphe orienté

Dans un graphe orienté :

- Les arêtes de l'arbre BFS sont des arêtes d'arbre.
- Les autres arêtes sont soit :
 - des **arêtes arrière** (vers un ancêtre),
 - soit des **arêtes croisées (transversales)**.

La BFS ne produit pas d'arêtes avant au sens DFS.

Propriétés fondamentales de la BFS

Soit G un graphe (orienté ou non) et une BFS lancée à partir d'un sommet s .

1 - Accessibilité

La BFS visite **tous les sommets accessibles depuis s** .

Autrement dit, elle explore exactement la composante atteignable à partir de s .

2 - Propriété de plus court chemin

Si un sommet v est au **niveau i** dans l'arbre BFS, alors :

- le chemin dans l'arbre BFS entre s et v contient exactement i arêtes ;
- tout autre chemin de s à v contient **au moins i arêtes**.

👉 Donc l'arbre BFS encode les **plus courts chemins en nombre d'arêtes**.

3 - Structure des niveaux

Si (u, v) est une arête du graphe qui n'est pas une arête d'arbre BFS, alors :

$$\text{niveau}(v) \leq \text{niveau}(u) + 1$$

Autrement dit, une arête ne peut relier que :

- deux sommets du même niveau,
- ou deux niveaux consécutifs.

Elle ne peut jamais "sauter" plusieurs niveaux.

Analyse de complexité, exploration et reconstruction d'un chemin avec la BFS

Soit :

- n le nombre total de sommets,
- m le nombre total d'arêtes,
- n_s le nombre de sommets accessibles depuis s ,
- m_s le nombre d'arêtes incidentes à ces sommets.

La complexité d'une BFS est :

$$O(n_s + m_s)$$

et dans le pire cas :

$$O(n + m)$$

Chaque sommet est inséré dans la file au plus une fois,
et chaque arête est examinée au plus une fois (ou deux fois en non-orienté).

Exploration complète du graphe

Si le graphe n'est pas connexe, la BFS lancée à partir de s n'explore que la composante atteignable depuis s .

Pour explorer tout le graphe, on peut :

- relancer la BFS à partir d'un sommet non visité,
- exactement comme pour la DFS.

Reconstruction d'un chemin

À l'aide du tableau `discovered` (ou des parents),
on peut reconstruire le plus court chemin entre s et un sommet v en remontant les prédécesseurs.

Conclusions

- Nous avons implémenté AdjacencyMapGraph
- Nous avons regardé 2 méthodes de parcours : en profondeur et en largeur.
- Nous avons regardé les propriétés de ces 2 méthodes.